

Rust — A Complete Reference for Python Developers

A practical guide to the Rust programming language, written for engineers coming from Python.

Preface

Who this document is for

This document is written for a working Python developer — particularly one doing data, backend, or AI/ML engineering — who needs to become genuinely productive in Rust rather than merely conversant with its syntax.

It assumes you can already program well. It does not assume you have ever thought about the stack, the heap, or who is responsible for freeing memory, because Python never asked you to.

How it is organised

Every concept is presented three times over, at increasing depth:

1. **What Rust does** — the rule, the syntax, the mechanics.
2. **How it differs from Python** — the intuition you must actively unlearn, stated explicitly rather than left for you to discover in a compiler error.
3. **Why Rust does it that way** — the design constraint the rule falls out of. Almost every "arbitrary" rule in Rust is a consequence of one decision: *the compiler, not a runtime, decides when memory is freed*.

The material is sequenced so that each part depends only on the parts before it. Chapters within a part are self-contained enough to be used as reference after a first read.

Each part closes with exercises. They are not decoration — Rust is one of the few languages where reading is a genuinely poor substitute for compiling. Several exercises deliberately ask you to write code that *fails*, because the compiler's error message is frequently the clearest explanation of the rule available.

Conventions

-  marks code that compiles;  marks code that does not.
 - Code targets **Rust edition 2024** (the default since Rust 1.85) on current stable Rust.
 - Where a feature has a stabilisation version worth knowing, it is stated inline.
 - Crate recommendations name the current community-standard choice, not an exhaustive list.
-

Table of Contents

Part I — Getting Started

1. The Mental Shift from Python
2. Toolchain and Project Setup
3. The Development Loop
 - Exercises — Part I

Part II — Language Foundations

1. Variables and Bindings
2. Scalar Types
3. Compound Types
4. Strings
5. Type Inference and Casting
6. Functions and Control Flow
7. Structs and Methods
8. Enums and Pattern Matching
9. Modules, Crates, and Visibility
 - Exercises — Part II

Part III — Ownership, Borrowing, and Errors

- Memory layout: a worked walkthrough
1. Ownership
 2. Borrowing and References
 3. Slices
 4. Lifetimes
 5. Error Handling
 6. Working with the Borrow Checker
 - Exercises — Part III

Part IV — Abstraction

1. Generics
2. Traits
3. Static and Dynamic Dispatch
4. Closures
5. Iterators
6. Collections
7. Smart Pointers and Interior Mutability
 - Worked example
 - Exercises — Part IV

Part V — Testing, Documentation, and Tooling

1. Testing
2. Documentation
3. Debugging and Diagnostics
4. Cargo in Depth
 - Exercises — Part V

Part VI — Memory Safety and the Edges of the Type System

1. What Memory Safety Means
2. `unsafe`
3. Advanced Lifetimes
4. Macros
5. Const Generics
 - Exercises — Part VI

Part VII — Concurrency and Async

1. Threads
2. `Send` and `Sync`
3. Shared State
4. Message Passing
5. Atomics
6. Data Parallelism with Rayon
7. Async
8. Choosing the Right Tool
 - Exercises — Part VII

Part VIII — Practice

1. Performance
2. Python-Developer Gotchas
3. The Ecosystem
4. Numerical, Data, and ML Work
5. A Learning Path
6. Further Reading

Appendix A — Python to Rust Quick Reference Appendix B — Glossary

Part I — Getting Started

1. The Mental Shift from Python

1.1 The one difference everything else follows from

In Python, a variable is a **name pointing at an object on the heap**. The object knows its own type at runtime, owns its own memory, and the garbage collector frees it when no names point to it. Types are checked while the program runs.

In Rust, a variable is a **binding to a value with a fixed, compile-time-known type and a known size**. There is no garbage collector and no runtime type information. The compiler must know, before the program runs, exactly how many bytes each value occupies and exactly when it is freed.

Almost every rule in this document falls out of that one constraint.

Two consequences worth holding in mind from the start:

1. **Types are erased at runtime.** `i32`, `String`, your own structs — none of them carry a type tag at runtime the way a Python object does. The type exists only to instruct the compiler. This is why Rust is fast and why the compiler is strict.
2. **Size must be known at compile time** for values living on the stack. This is why `String` and `&s` `tr` are different types, why arrays carry their length in the type, and why `Box<T>` exists.

1.2 The question Rust is answering

Every language must answer: **when is a value's memory freed, and who decides?**

Language family	Who decides	Cost
C / C++	You, manually	Fast — and the source of most serious security vulnerabilities of the last forty years: use-after-free, double-free, dangling pointers, data races
Python / Java / Go	A runtime — reference counting and/or a tracing garbage collector	Safe — but costs a runtime, unpredictable pauses, and memory overhead
Rust	The compiler, at compile time , using rules encoded in the types	Zero runtime cost; unsafe patterns are rejected before the program runs

Ownership is that third answer. It is not a memory-management feature bolted onto the language — it is the language. Lifetimes, `&`/`&mut`, `String` versus `&str`, iterator semantics, and `Send`/`Sync` are all the same idea viewed from different angles.

1.3 A stack and heap primer

Python hides this from you completely. Rust makes it central, so it is worth stating plainly before any types are introduced.

- **Stack** — fast, LIFO, automatically managed. Stores values whose size is known at compile time. Pushing and popping is just moving a pointer.
- **Heap** — for data whose size is dynamic or large. Slower, and requires bookkeeping. In Rust there is no garbage collector; the compiler inserts the cleanup based on ownership rules.

The working rule: **scalar types, fixed-size arrays, and tuples of fixed-size things live on the stack**. Growable things such as `String` and `Vec<T>` keep a small fixed-size *handle* on the stack — pointer, length, capacity — and the actual data on the heap.

That single fact explains the `String` versus `&str` distinction, the move semantics of Chapter 13, and most of what follows.

1.4 Expectations

The difficulty of Rust is concentrated, not spread out. Part III is hard; almost nothing after it is. Expect roughly two weeks during which the borrow checker rejects code you are certain is correct, followed by a period in which you stop noticing it at all — while it continues catching bugs Python would have shipped.

2. Toolchain and Project Setup

2.1 Installing

Install via `rustup`, the official toolchain manager, from `rustup.rs`. It manages compiler versions, targets, and components, and is the only installation method worth using.

```
rustup update           # update the toolchain
rustup component add clippy rustfmt
rustc --version
cargo --version
```

`rustup` installs `rustc` (the compiler), `cargo` (build tool and package manager), `clippy` (the linter), and `rustfmt` (the formatter).

2.2 Creating a project

```
cargo new my_app        # binary project
cargo new my_lib --lib   # library project
```

```
my_app/
├─ Cargo.toml           # manifest: metadata + dependencies
├─ Cargo.lock           # exact resolved versions
└─ src/
    └─ main.rs          # entry point
```

`Cargo.toml` is the manifest:

```
[package]
name = "my_app"
version = "0.1.0"
edition = "2024"

[dependencies]
serde = { version = "1", features = ["derive"] }

[dev-dependencies]           # test and benchmark only; not shipped
criterion = "0.5"
```

Put logic in `lib.rs` and keep `main.rs` a thin wrapper from the start. A project may contain both a library crate and a binary crate. Code in the library is testable by integration tests and reusable; code in `main.rs` is neither. This costs nothing to do on day one and is disruptive to retrofit later.

2.3 Editions

Rust guarantees backward compatibility, so opt-in language changes ship as **editions**: 2015, 2018, 2021, and 2024. The edition is set per-crate in `Cargo.toml`, and crates of different editions interoperate freely — a 2015-edition dependency works in a 2024-edition project.

Use `edition = "2024"` for new projects. Be aware that older tutorials may show idioms that are now outdated (`extern crate`, pre-2018 module paths, bare trait objects without `dyn`). If a tutorial's syntax looks archaic, check its date.

2.4 rust-analyzer

Install the **rust-analyzer** extension for VS Code, or the equivalent LSP client for your editor. It provides inline type hints for every inferred binding, inline error display, go-to-definition into the standard library source, and auto-import.

Rust without rust-analyzer is substantially harder than Rust with it. The inline type annotations alone teach inference by continuously showing you what the compiler concluded. If you are evaluating whether Rust is tolerable to work in, evaluate it with rust-analyzer running.

2.5 Debug versus release builds

This is the single most common false alarm for newcomers, so it belongs on page one rather than in a performance chapter.

Debug builds are 10–100× slower than release builds. That is not a typo.

Debug builds use `opt-level = 0`, enable integer overflow checks and debug assertions, and perform no inlining. If you benchmark a Rust program with `cargo run` and conclude it is slower than the equivalent NumPy, this is why.

```
cargo run --release          # ALWAYS, for any performance measurement
```

Development is fast to compile and slow to run; production is the reverse. If your debug build is too slow to test with — common with image or tensor work — optimise only your dependencies and keep your own code debuggable:

```
[profile.dev.package."*"]
  opt-level = 3
```

3. The Development Loop

3.1 The commands that matter

```
cargo check                  # type-check WITHOUT code generation — 5-10× faster than build
cargo build                  # debug build
cargo build --release        # optimised build
cargo run -- arg1 arg2      # run; arguments after --
cargo test
cargo clippy
cargo fmt
cargo add serde --features derive # add a dependency (built in since 1.62)
cargo tree                   # dependency graph; find duplicate versions
cargo doc --open             # build and view docs for your crate and all dependencies
```

Internalise `cargo check`. Your inner loop is `cargo check`, not `cargo build`. rust-analyzer runs it continuously in the background as you type.

3.2 Clippy

```
cargo clippy
cargo clippy -- -W clippy::pedantic      # much stricter
cargo clippy --fix                        # auto-apply the safe fixes
```

Clippy ships 700+ lints across correctness, performance, style, and complexity. Critically, **it does not merely flag problems — it names the idiomatic alternative**. It will tell you to use `if let` instead of a two-arm `match`, that `.iter().count()` should be `.len()`, that a `&String` parameter should be `&str`, and that your manual accumulation loop is a `fold`.

Run `cargo clippy` on every piece of code you write, from the first day. It converts "code that compiles" into "code a Rust developer would write" faster than any book, and it is the single highest-leverage habit available to a Rust beginner.

`cargo fmt` runs rustfmt: non-negotiable formatting with no configuration debates, universally adopted across the ecosystem. Comparable to `black`, but with no competing alternatives.

3.3 Reading compiler errors

Rust's diagnostics are the best in the industry. They name the problem, point at the exact source span, explain the rule being violated, and usually suggest a concrete fix.

```
rustc --explain E0502      # a full essay on any error code
RUST_BACKTRACE=1 cargo run # stack trace on panic
RUST_BACKTRACE=full cargo run
```

Coming from Python tracebacks, the instinct is to skim the error and start guessing. Resist it. In Rust, **the error message is the lesson** — reading it in full is usually faster than any other approach, and it is how most of the borrow checker's rules are actually learned.

Exercises — Part I

1. **Set up a project.** Run `cargo new scratch`, then `cargo run`. Add a second file `src/lib.rs` containing one public function, call it from `main.rs`, and confirm it builds.
2. **Feel the difference.** Write a loop summing the integers from 1 to 500 000 000. Time it under `car go run` and again under `cargo run --release`. Record the ratio; this number is worth remembering.
3. **Meet clippy.** Write a deliberately clumsy function — a manual index loop that could be an iterator, a `&String` parameter, an `if x == true`. Run `cargo clippy` and apply every suggestion.
4. **Read an error code.** Introduce a type error on purpose, note the `E0308`-style code in the output, and run `rustc --explain` on it.

5. **Check versus build.** Time `cargo check` against `cargo build` on a project with a few dependencies. Confirm the difference for yourself.

Part II — Language Foundations

4. Variables and Bindings

4.1 `let` and immutability by default

```
let x = 5;           // x is bound to 5; type inferred as i32
x = 6;               // ❌ cannot assign twice to immutable variable
```

In Python everything is mutable by default and you opt into immutability through tuples, frozensets, and naming conventions. Rust inverts this: **bindings are immutable unless declared otherwise.**

```
let mut x = 5;       // explicitly mutable
x = 6;               // ✅
```

This is not a style preference. Immutability is the default the compiler optimises around and the borrow checker reasons about.

`mut` is part of the *binding*, not the type. It permits the value behind the name to change — either by assignment, or by calling a method that mutates it.

4.2 Shadowing

```
let x = 5;
let x = x + 1;           // new binding; shadows the old x
let x = x * 2;           // and again
let x = "now a str";     // the TYPE may change
```

This resembles Python reassignment but is fundamentally different. Each `let x` creates a **brand-new variable** that reuses the name. The old value still exists until its scope ends; you have simply lost the name for it. Because it is a new binding, the type is free to change.

Shadowing lets you transform a value through stages under one clean name, *without* making the binding

`mut`:

```
let spaces = "  "; // &str
let spaces = spaces.len(); // usize - same name, different type
```

In Python you would reassign and the type would silently change anyway. In Rust this is an explicit, scoped, compiler-visible act — and it is not mutation.

4.3 Scope and block expressions

Rust scopes are lexical and block-based, but **a block is an expression that evaluates to a value**:

```
let y = {
    let a = 3;
    let b = 4;
    a * a + b * b // no semicolon → this is the block's value
}; // y == 25
```

The missing semicolon is significant. An expression *without* a trailing semicolon is the value of the block; *with* a semicolon it becomes a statement evaluating to `()` — the **unit type**, Rust's "nothing". Unit is a real zero-sized type, loosely analogous to Python's `None` but not a value you check for.

This distinction causes early confusion and is worth internalising now, because it recurs in every function body, `if`, `match`, and `loop`.

4.4 `const` and `static`

```
const MAX_TOKENS: usize = 8192; // compile-time constant
static GREETING: &str = "hello"; // fixed memory location for the whole program
```

	<code>const</code>	<code>static</code>
Type annotation	required	required
Evaluation	inlined at compile time	lives at a fixed address for the whole run
Mutable?	never	only via <code>static mut</code> , which is <code>unsafe</code> and discouraged
Analogy	a literal substituted everywhere it appears	a single global object

Both must be initialised to a value computable at compile time. The naming convention is `SCREAMING_SNAKE_CASE`, and unlike `let`, the type annotation is mandatory — the compiler does not infer types for items at module level.

5. Scalar Types

Rust has four scalar categories: integers, floating-point numbers, booleans, and characters.

5.1 Integers

Python has a single unbounded `int`. Rust has **twelve** integer types, and you must choose one — or let inference choose `i32`, the default.

Width	Signed	Unsigned
8-bit	<code>i8</code>	<code>u8</code>
16-bit	<code>i16</code>	<code>u16</code>
32-bit	<code>i32</code> (default)	<code>u32</code>
64-bit	<code>i64</code>	<code>u64</code>
128-bit	<code>i128</code>	<code>u128</code>
pointer-sized	<code>isize</code>	<code>usize</code>

- **Signed** (`iN`) range: -2^{n-1} to $2^{n-1}-1$. **Unsigned** (`uN`) range: 0 to 2^n-1 .
- `usize` and `isize` are the width of a pointer on the target machine — 64 bits on a typical 64-bit build. `usize` is the type of every index and every length in Rust. Indexing a `Vec`, calling `.len()`, slicing — all `usize`.
- `u8` doubles as Rust's raw byte. `Vec<u8>` is the idiomatic "bag of bytes".

Integer literals are flexible:

```
let a = 98_222;      // _ is a visual separator
let b = 0xff;        // hex
let c = 0o77;        // octal
let d = 0b1010_0011; // binary
let e = b'A';        // byte literal → u8 value 65
let f = 42u64;       // type suffix
let g: i64 = 42;     // or annotate the binding
```

Overflow. Because integers are fixed-width, they can overflow, and Rust's behaviour is build-mode-dependent:

- **Debug builds** — overflow **panics**. This is a feature; it catches bugs loudly.
- **Release builds** — overflow **wraps** using two's complement (`255u8 + 1 == 0`), silently, for speed.

Relying on silent wrapping is a latent bug, so Rust provides explicit methods for stating the behaviour you actually want:

```
let x: u8 = 255;

x.wrapping_add(1);    // 0           - wrap deliberately
x.checked_add(1);     // None        - Option; Some(v) if it fits
x.saturating_add(1);  // 255        - clamp at the maximum
x.overflowing_add(1); // (0, true)   - value plus a did-it-overflow flag
```

This is a recurring theme: where Python picks one behaviour for you, Rust makes you name the behaviour you want.

5.2 Floating point

Two types, both IEEE 754:

```
let x = 2.0;          // f64 - the default
let y: f32 = 3.0;     // single precision
```

`f64` is the default because on modern hardware it is roughly as fast as `f32` and more precise. Numerical and ML work deliberately reaches for `f32` — and for `f16` / `bf16` via crates — when memory bandwidth or model size dominates, but the language default is `f64`.

The usual floating-point caveats apply and Rust does not hide them: `0.1 + 0.2 != 0.3`, and `NaN != NaN`. The latter means floats are only *partially* ordered, which is why they cannot be used directly as `HashMap` keys or sorted with `.sort()`. Special values are available as `f64::NAN`, `f64::INFINITY`, and so on.

5.3 Booleans

```
let t = true;
let f: bool = false;
```

One byte in size. The rule that matters: **Rust has no truthiness**. `if x` requires `x: bool`. There is no "0 is falsy", no "empty string is falsy", no "None is falsy". `if some_vec` is a compile error; you write `if !some_vec.is_empty()`. This eliminates an entire class of Python bug.

5.4 Characters

```
let c = 'z';
let heart = '♥';
let crab = '🦀';
```

A Rust `char` is **four bytes** and represents a single **Unicode scalar value** — a code point, not a byte.

- Python's `str` is a sequence of code points; iterating yields one-character strings. Rust's `char` is the closest analogue to one of those.
- But a `String` is not a sequence of `char` in memory — it is UTF-8 bytes. A `char` occupies 1–4 bytes once encoded into a string. This mismatch is exactly why a `String` cannot be indexed by integer position (Chapter 7).

Single quotes produce a `char`; double quotes produce a string. They are different types, unlike Python where `'a'` and `"a"` are identical.

6. Compound Types

6.1 Tuples

Fixed-length, ordered, **heterogeneous**, known size, stack-allocated.

```
let tup: (i32, f64, char) = (500, 6.4, 'x');

let (a, b, c) = tup;           // destructuring
let first = tup.0;             // access by index with .N — not [0]
let second = tup.1;
```

Differences from Python tuples:

- Access is `.0`, `.1`, `.2` — a compile-time field access, not runtime indexing. A tuple cannot be indexed by a variable, because each position has a different type and the type must be known at compile time.
- The **empty tuple** `()` is the *unit type* — Rust's "no meaningful value". Functions that return nothing return `()`. It is a real zero-sized type, not a null.
- The standard library implements the common traits on tuples only up to twelve elements. Beyond that, use a struct.

6.2 Arrays

Fixed-length, **homogeneous**, known size, stack-allocated. **The length is part of the type.**

```
let arr: [i32; 5] = [1, 2, 3, 4, 5]; // the type is literally "[i32; 5]"
let zeros = [0; 5];                 // [0, 0, 0, 0, 0] — value ; count
let x = arr[0];                     // indexing with usize
```

- `[i32; 5]` and `[i32; 6]` are **different types**. This is nothing like a Python list.

- Indexing is **bounds-checked at runtime**. `arr[10]` compiles but panics with "index out of bounds" rather than reading arbitrary memory as C would. Safety at a very small cost, which the optimiser frequently eliminates entirely.
- Arrays do not grow. The growable, heap-backed equivalent of a Python list is `Vec<T>` (Chapter 24). Mentally: *array = fixed stack buffer*, `Vec` = *dynamic heap list*.
- A **slice** `&[T]` is a borrowed view into a contiguous run of elements — a pointer plus a length, owning nothing. Slices are the bridge that lets one function accept both arrays and vectors (Chapter 15).

7. Strings

Python has one `str`. Rust has two string types you meet immediately, and the distinction is the most common early stumbling block. It exists entirely because of the stack/heap and ownership model.

7.1 `&str` versus `String`

```
let a: &str = "hello";           // string literal — a &str
let b: String = String::from("hi"); // owned, heap-allocated, growable
let c: String = "hi".to_string();  // the same thing, different constructor
```

	<code>&str</code>	<code>String</code>
Owns its data?	No — it <i>borrow</i> s a view	Yes — owns heap memory
Growable?	No	Yes (<code>push</code> , <code>push_str</code> , <code>+</code>)
Where the bytes live	Could be the binary, a <code>String</code> , anywhere	The heap
Stack footprint	pointer + length	pointer + length + capacity
Python analogy	A read-only window onto text	A real, mutable string object

The mental model: **`String` is the owner; `&str` is a borrowed window onto some UTF-8 bytes** — which may be owned by a `String`, or baked into the compiled binary in the case of a literal.

Converting `String` → `&str` is free (`&my_string`); converting `&str` → `String` allocates, so Rust makes it explicit (`.to_string()`).

7.2 UTF-8 and why indexing is forbidden

Both types are **guaranteed valid UTF-8**. This is why a string cannot be indexed by integer:

```
let s = String::from("héllö");
let c = s[0];      // ❌ strings are not indexable by usize
```

Because a character may occupy multiple bytes in UTF-8, "the byte at position 0" and "the first character" are different questions, and Rust refuses to let you conflate them. You state your intent instead:

```
for ch in s.chars() { /* iterate Unicode scalar values */ }
for b in s.bytes() { /* iterate raw u8 bytes */ }
let slice = &s[0..2]; // byte-range slice – panics if it splits a char boundary
```

The essential takeaway: **literal** = `&str` (borrowed); **owned and growable** = `String`; **both are UTF-8; no integer indexing.**

8. Type Inference and Casting

8.1 Inference

Rust performs powerful **local type inference**, so annotations are frequently unnecessary:

```
let x = 5; // inferred i32
let v = vec![1, 2, 3]; // inferred Vec<i32>
```

Inference is local and bidirectional, and occasionally there is not enough information — most famously when a method could produce many types:

```
let guess = "42".parse().unwrap(); // ❌ parse into WHAT?
let guess: u32 = "42".parse().unwrap(); // ✅ annotation resolves it
let guess = "42".parse::<u32>().unwrap(); // ✅ or annotate the call
```

The `::<u32>` syntax is called the **turbofish**. It appears wherever the return type cannot be inferred from context.

8.2 Casting with `as`

Rust performs **no implicit numeric coercion**. Adding a `u8` to a `u32` does not compile. Unlike Python, where ints and floats mix freely, every conversion is visible:

```
let a: i32 = 1000;
let b = a as i64; // widening – always safe
let c = a as u8; // narrowing – truncates: 1000 as u8 == 232
let d = 3.9_f64 as i32; // float → int truncates toward zero → 3
```

`as` is the blunt instrument: fast, but it truncates on narrowing without complaint. For safe, fallible conversions, use the `From` and `TryFrom` traits (Chapter 20).

For now: `as` exists, there is **no automatic widening**, and the compiler forcing you to write the conversion is the language preventing a silent precision bug.

9. Functions and Control Flow

9.1 Functions

```
fn add(x: i32, y: i32) -> i32 {  
    x + y                                // no semicolon = the return value  
}  
  
fn early(x: i32) -> i32 {  
    if x < 0 { return 0; }               // explicit return also works  
    x * 2  
}
```

Parameter types and return types are always explicit — Rust never infers a function signature. Inside the body, inference does the rest.

9.2 Everything is an expression

There is no ternary operator in Rust because `if` already is one:

```
let grade = if score > 90 { "A" } else { "B" };    // if as expression  
let x = match n { 0 => "zero", _ => "other" };      // match as expression  
let y = loop { break 42; };                       // loop returns a value
```

Both branches of an `if` used as an expression must have the same type.

9.3 Loops

Three forms. `loop` is the only one that can return a value, via `break`:


```

loop { /* ... */ }                // infinite; break with a value
while cond { /* ... */ }
for x in collection { /* ... */ }  // the one used 95% of the time

'outer: for i in 0..10 {           // labelled loops for nested breaks
    for j in 0..10 {
        if done { break 'outer; }
    }
}

```

Ranges: `0..5` is exclusive of the endpoint, `0..=5` inclusive. There is no C-style `for (;;) loop` — write `for i in 0..n`.

10. Structs and Methods

10.1 The three struct forms

```

struct Model {                    // named-field struct
    name: String,
    layers: u32,
}

struct Point(f64, f64);          // tuple struct — access with .0, .1
struct Marker;                   // unit struct — zero size, useful as a type
tag

```

10.2 `impl` blocks

```

impl Model {
    // associated function (no self) — the constructor convention
    fn new(name: &str, layers: u32) -> Self {
        Self { name: name.to_string(), layers }    // field init shorthand
    }

    fn describe(&self) -> String { /* ... */ }      // borrows
    fn add_layer(&mut self) { self.layers += 1; }   // mutably borrows
    fn consume(self) -> String { self.name }        // takes ownership
}

```

Points that matter to a Python developer:

- `Self` (capitalised) means "this type"; `self` (lowercase) is the instance. `Self { .. }` is the idiomatic constructor body.
- **There is no `__init__`.** `new()` is pure convention, not a language feature. A type may have `from_config()`, `with_capacity()`, `parse()`, and any number of other constructors.
- **The three receivers matter enormously:** `&self` to read, `&mut self` to write, `self` to consume. This is ownership expressed in method signatures, and choosing the wrong one is the leading cause of borrow errors in your own APIs.

10.3 Struct update syntax

Rust has no keyword arguments and no default parameter values. Struct update syntax fills part of that gap:

```
let cfg = Config { layers: 24, ..Default::default() };
```

The remaining approaches to optional configuration are covered in Chapter 44.

11. Enums and Pattern Matching

11.1 Enums are algebraic data types

Python's `Enum` is a set of named constants. Rust's enums are far more capable — **each variant may carry different data**:

```
enum Message {  
    Quit,                                // no data  
    Move { x: i32, y: i32 },             // struct-like variant  
    Write(String),                       // tuple-like variant  
    ChangeColor(u8, u8, u8),  
}
```

This is the single most useful modelling tool in the language. `Option<T>` and `Result<T, E>` are ordinary enums with no special compiler support.

The Python equivalent requires a class hierarchy plus `isinstance` checks, with no exhaustiveness guarantee. **Model your domain with enums** and the compiler will force every case to be handled, permanently.

11.2 `match`

```
match msg {
    Message::Quit => { /* ... */ }
    Message::Move { x, y } => { /* ... */ }           // destructure a struct variant
    Message::Write(text) if text.is_empty() => { /* ... */ } // match guard
    Message::Write(text) => { /* ... */ }
    Message::ChangeColor(r, g, b) => { /* ... */ }
}

match n {
    0 => "zero",
    1..=9 => "digit",           // range pattern
    x if x < 0 => "negative",    // guard
    _ => "big",                 // catch-all
}
```

Patterns work outside `match` too:

```
let (a, b, ..) = tuple;           // .. ignores the rest
let Point { x, .. } = point;      // partial destructure
let msg @ 1..=5 = n;              // @ binds AND tests
```

11.3 Exhaustiveness

`match` is **exhaustive**. Omitting a variant is a compile error, not a silent fall-through.

This is one of Rust's quiet superpowers. Add a variant to an enum six months later, and the compiler shows you every single place that needs updating. Refactoring in Rust is qualitatively different from refactoring in Python because of this property.

11.4 The three shorthands

```
if let Some(x) = opt { /* ... */ } else { /* ... */ } // one case matters
while let Some(x) = stack.pop() { /* ... */ }         // loop until None
let Some(x) = opt else { return; };                   // let-else: bind or diverge
```

`let ... else` (stable since Rust 1.65) is the cleanest way to write guard clauses and dramatically reduces nesting. It is consistently underused by newcomers.

12. Modules, Crates, and Visibility

You need this the moment `main.rs` exceeds a couple of hundred lines — which is typically day two.

12.1 Layout

```
my_project/
├─ Cargo.toml
└─ src/
    ├─ main.rs          # binary crate root
    ├─ lib.rs           # library crate root
    ├─ config.rs        # a module
    └─ model/
        ├─ mod.rs       # module root (or model.rs alongside the model/ directory)
        └─ layers.rs    # submodule
```

```
// main.rs
mod config;           // declares and loads src/config.rs
mod model;            // loads src/model/mod.rs or src/model.rs

use config::Settings; // bring a name into scope
use model::layers::Dense;
```

```
// config.rs
pub struct Settings { pub dim: u32, dropout: f32 } // per-FIELD visibility
pub fn load() -> Settings { /* ... */ }
fn helper() { }                                     // private to this module
```

12.2 Visibility

Modifier	Visible to
<i>(none)</i>	The current module and its children
<code>pub</code>	Everyone
<code>pub(crate)</code>	Anywhere in this crate — the one you will want most often
<code>pub(super)</code>	The parent module

Path prefixes: `crate::` (crate root), `super::` (parent module), `self::` (current module).

Everything is private by default, including struct fields. Unlike Python's `_private` naming convention, this is enforced by the compiler rather than suggested. Encapsulation in Rust is real.

12.3 Crates, packages, and workspaces

- A **crate** is a compilation unit — one library or one binary.
- A **package** is one `Cargo.toml`, containing at most one library crate and any number of binary crates.
- A **workspace** is a monorepo of related packages sharing one `Cargo.lock` and one `target/` directory:

```
[workspace]
members = ["core", "cli", "python-bindings"]
```

This is the standard layout for a project with Python bindings: a pure Rust core crate, a thin binding crate, and a CLI.

Exercises — Part II

1. **Overflow modes.** Make a `u8` equal to 250, then add 10 to it four ways — `wrapping_add`, `checked_add`, `saturating_add`, `overflowing_add`. Print each result and explain the difference in a comment.
2. **Shadowing versus `mut`.** Write a snippet that takes a `&str` of digits, shadows it into the parsed number, then shadows again into the number doubled — without ever using `mut`.
3. **The unit type.** Write a block expression that computes a value, and a second block that ends in a semicolon. Bind both with `let` and use the compiler errors to discover the type of the second.
4. **String reality.** Take `let s = String::from("hello");`. Print `s.len()` and `s.chars().count()` and explain why they differ.
5. **No implicit coercion.** Try to add an `i32` and an `i64` directly. Read the error, fix it with `as`, then fix it again by changing a type annotation instead. Note that both fixes are valid and that they mean different things.
6. **Turbofish.** Parse `"3.14"` into an `f64` two ways: with a binding annotation, and with the turbofish.
7. **Expression orientation.** Rewrite an `if/else` that assigns to a mutable variable so that it assigns once from an `if` expression instead. Then do the same with a `match`.
8. **Model with an enum.** Define an enum with at least one unit variant, one tuple variant, and one struct variant. Write an exhaustive `match` over it. Add a fourth variant and observe every error the compiler reports.
9. **Guards and `let ... else`.** Rewrite a nested `if let` chain using `let ... else` guard clauses and compare the nesting depth.
10. **Split into modules.** Take a single-file program of at least 150 lines and split it into three modules. Make exactly one item `pub`, one `pub(crate)`, and leave one private. Confirm you get an error when you use the private one from another module.

Part III — Ownership, Borrowing, and Errors

This is the part where Python intuition actively works against you, and the part that makes everything after it straightforward.

Memory layout: a worked walkthrough

Before the ownership rules, it is worth seeing exactly where bytes live. Every rule in this part is easier to accept once the picture is concrete.

All addresses and sizes below are real output from a 64-bit build on Rust 1.96.

The two-line program

```
let mut name = String::from("Deepak");
name.push_str("Kumar G");
```

Step 0 — the literal is not on the stack or the heap

"Deepak" in the source is a `&'static str`. Its bytes are compiled **into the executable**, in the read-only data segment. They exist before `main` runs and are never freed.

```
BINARY (.rodata, read-only)
```

44 65 65 70 61 6B	"Deepak"
-------------------	----------

▲ address 0x7ff782dfd47f

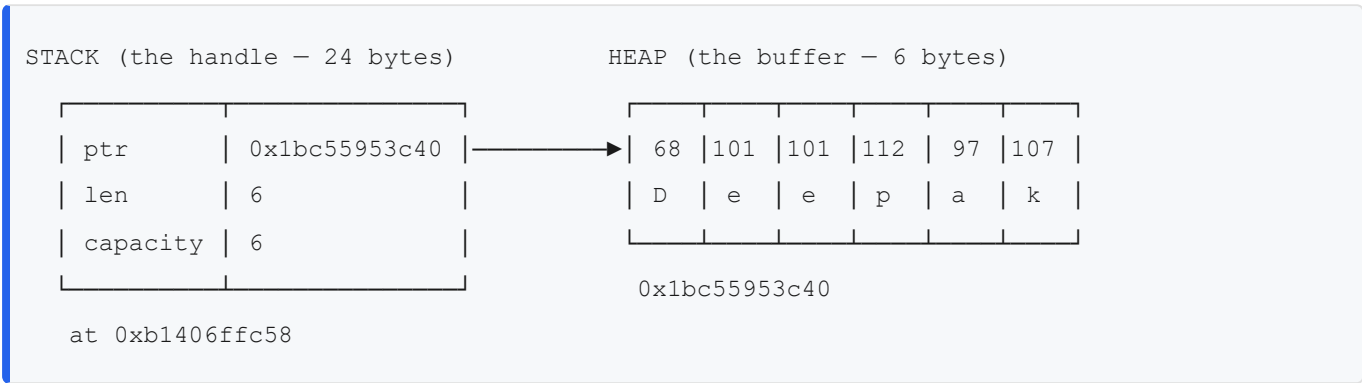
A `&str` is a **fat pointer** — two words, a pointer and a length — and it owns nothing:

```
STACK
```

ptr	0x7ff782dfd47f	→ into .rodata
len	6	

This is the first correction to the common mental model: **a string literal touches neither the stack nor the heap**. It lives in the binary.

Step 1 — `String::from("Deepak")` allocates and copies

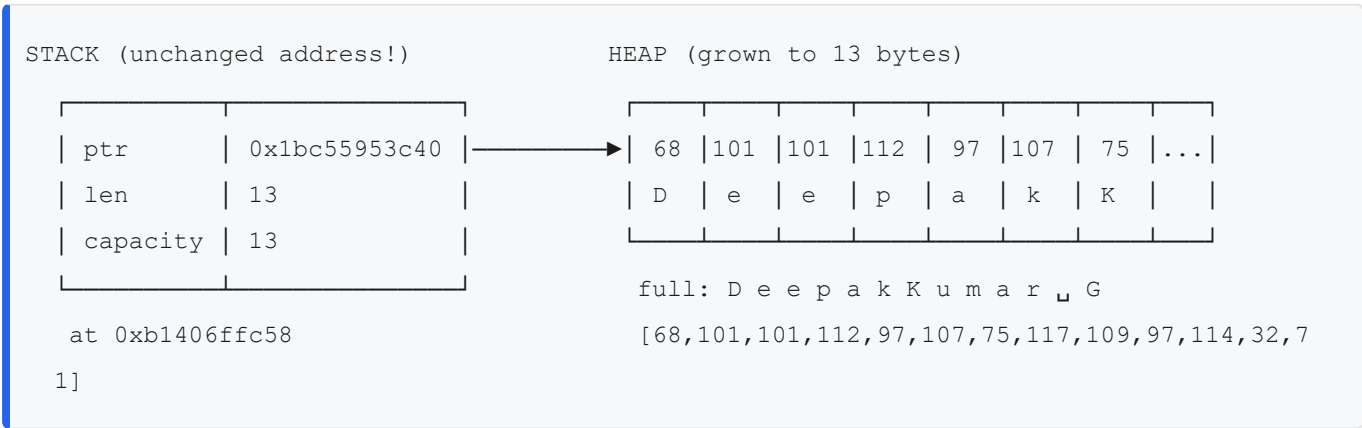


Three facts to read off this diagram:

- `size_of::<String>()` is **24 bytes** — three 8-byte words — no matter how long the text is. A one-character `String` and a one-gigabyte `String` have identically sized handles.
- `String::from` **allocates exactly** `len`, so capacity is 6, not rounded up.
- The literal's bytes were **copied** from `.rodata` to the heap. This is why `&str → String` allocates and is explicit, while `String → &str` is free.

Step 2 — `push_str("Kumar G")` needs 13 bytes but has room for 6

`"Kumar G"` is 7 bytes, so the required length is 6 + 7 = 13, which exceeds the capacity of 6. The buffer must be **reallocated**.



allocates elsewhere, copies the bytes, and frees the old block. On the run above, the Windows system allocator extended in place, so the pointer stayed at `0x1bc55953c40`.

This is not something you may rely on. It is allocator- and fragmentation-dependent, and Rust's borrow checker correctly assumes the buffer *can* move. That assumption is exactly why §15 forbids holding a slice across a mutating call.

Step 3 — the growth strategy

`push_str` does not reallocate on every call. Capacity grows amortised, roughly doubling:

```
len 1 → capacity 8      (minimum non-zero capacity for byte-sized elements)
len 9 → capacity 16
len 17 → capacity 32
len 33 → capacity 64
```

The exception is a single large append like ours, where the required length (13) exceeds double the old capacity (12), so capacity becomes exactly the required 13. `Vec<T>` uses the identical strategy — `String` is a `Vec<u8>` with a UTF-8 guarantee.

Use `String::with_capacity(n)` or `Vec::with_capacity(n)` when the final size is known; it turns several reallocations into one.

Step 4 — why this makes the borrow rule obvious

```
let mut name = String::from("Deepak");

let first = &name[0..6];           // borrows: points INTO the heap buffer
name.push_str("Kumar G");          // needs &mut name → may reallocate and free that buffer
println!("{}", first);              // would read freed memory
```

```
error[E0502]: cannot borrow `name` as mutable because it is also borrowed as immutable
  |
3 |     let first = &name[0..6];
  |               ---- immutable borrow occurs here
4 |     name.push_str("Kumar G");
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
5 |     println!("{}", first);
  |               ----- immutable borrow later used here
```

`first` is a `&str` pointing at `0x1bc55953c40`. If `push_str` reallocates and frees that block, `first` becomes a dangling pointer. In C this compiles and is a use-after-free. In Rust it is a compile error — and the rule that catches it is nothing more than "shared XOR mutable".

What actually lives where

The common belief that "`String` and `Vec` go on the heap; integers and floats go on the stack" is a good first approximation but is wrong in three ways worth fixing.

It is not the type that decides — it is whether the type owns an allocation, and where the owner itself lives.

Expression	Handle / value lives	Payload lives	Size of the handle
<code>let n: i32 = 42</code>	Stack	— (no heap at all)	4
<code>let f: f64 = 3.14</code>	Stack	—	8
<code>let a: [i32; 4]</code>	Stack	— (all four ints inline)	16
<code>let b = Box::new(99i32)</code>	Stack	Heap — an <code>i32</code> on the heap	8
<code>let v = vec![10, 20, 30]</code>	Stack	Heap — the <code>i32</code> s are on the heap	24
<code>let s = String::from("x")</code>	Stack	Heap	24
<code>"literal"</code>	—	Binary (.rodata)	16 (fat pointer)

The three corrections:

- Integers can absolutely be on the heap.** `Box<i32>` puts one there deliberately. Every element of a `Vec<i32>` is on the heap. The `i32` type has nothing to do with it — the *container* decides.
- A `String` field is not "on the heap" as a whole.** Its 24-byte handle sits inline wherever the struct sits; only the text bytes are on the heap:

```
struct User { id: u32, name: String, scores: Vec<f32> }
```

STACK — the whole User is 56 bytes, laid out inline

id: u32 (4 bytes + padding)

name: String ptr

scores: Vec<f32> ptr

24 bytes inline

24 bytes inline

HEAP: "Deepak"

HEAP: [1.0, 2.0]

If that `User` is itself inside a `Box` or a `Vec`, all 56 bytes are on the heap and the two buffers are separate heap allocations.

- String literals are in neither.** They are in the binary, as shown in Step 0.

A fourth point for completeness: a local variable may never touch the stack at all — the optimiser frequently keeps small values entirely in CPU registers. "Stack" in these diagrams means "not heap-allocated and scope-bound", which is the part that matters for ownership.

Other heap-owning types follow the same pattern: `Box<T>`, `Rc<T>`, `Arc<T>`, `HashMap`, `BTreeMap`, and `VecDeque` all keep a small handle where the variable lives and their payload on the heap.

Enums and `Option` in memory

An enum is a **tag plus enough space for its largest variant**:

```
enum Shape { Circle(f64), Rect { w: f64, h: f64 }, Empty }
```

```
size_of::<<Shape>>() == 24
```

tag (8)	payload: max(8, 16, 0) = 16
---------	-----------------------------

Circle, Rect, and Empty all occupy the same 24 bytes.

`Option<T>` is just an enum, but the compiler applies a **niche optimisation** where it can. If `T` has an impossible bit pattern — a `Box` or a `String` pointer can never be null — `None` is represented by that pattern and the tag costs nothing:

```
size_of::<<i32>>()           = 4
size_of::<<Option<i32>>>()   = 8    ← +4 for the tag (no spare bit pattern in an i32)

size_of::<<String>>()       = 24
size_of::<<Option<String>>>() = 24  ← FREE: None is a null pointer

size_of::<<Box<i32>>>()      = 8
size_of::<<Option<Box<i32>>>>() = 8  ← FREE
```

This is why `Option<T>` is not merely as safe as a nullable pointer but frequently **identical in memory** to one — while being impossible to dereference without checking. You get null-safety at literally zero cost.

Borrowing costs nothing

```
let r1: &String = &name;
let r2: &String = &name;
let slice: &str = &name[0..6];
```

STACK

```
name    at 0xb1406ffc58
r1      = 0xb1406ffc58
r2      = 0xb1406ffc58
```

both references hold the SAME address
8 bytes each, no copy, no allocation



slice ptr = 0x1bc55953c40, len = 6 → points DIRECTLY into the same heap buffer

A reference is one machine word. Borrowing copies no data and allocates nothing — which is why `&str` and `&[T]` are the correct parameter types, and why "borrow the input, own the output" costs nothing to follow.

13. Ownership

13.1 The three rules

Memorise these. Everything in this part is a consequence of them.

1. **Each value in Rust has a variable that is its *owner*.**
2. **There can be only one owner at a time.**
3. **When the owner goes out of scope, the value is *dropped*** — its memory is freed.

```
{
    let s = String::from("hello"); // s owns the heap buffer
    // ... use s ...
}
```

// s goes out of scope → drop(s) → heap freed

That closing brace is where the compiler inserts the deallocation. Not a garbage collector. Not you. The compiler, deterministically, at a location it knows at compile time.

This pattern — resource tied to scope, cleanup automatic — is called **RAII** (Resource Acquisition Is Initialization). It generalises beyond memory: files close, locks release, and sockets shut down at the closing brace too. The closest Python analogue is a `with` block, except that in Rust *every* value gets it, and it cannot be forgotten.

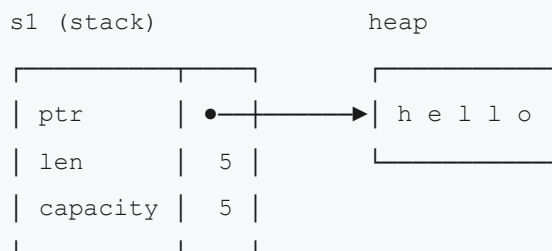
Drop order. Variables are dropped in **reverse declaration order**, like a stack unwinding; struct fields are dropped in **declaration order**. You can drop early with `drop(value)` — which is simply a function that takes ownership and does nothing, letting the scope rules do the work.

13.2 Move semantics

```
let s1 = String::from("hello");
let s2 = s1;
println!("{}", s1);    // ❌ borrow of moved value: `s1`
```

In Python, `s2 = s1` produces two names for one object. In Rust it **moves** ownership from `s1` to `s2`, and `s1` becomes statically invalid.

Mechanically: a `String` is three words on the stack — pointer, length, capacity — pointing at a heap buffer.



`let s2 = s1;` copies those three words — always a cheap, shallow, bitwise copy. Now **two owners point at one heap buffer**. When both go out of scope, both call `drop`, producing a **double free** and memory corruption.

Rust's fix is neither to deep-copy (expensive, and you did not ask for it) nor to reference-count (runtime cost). It is to declare `s1` **moved-out and unusable**. One owner, one drop. The problem is deleted rather than managed.

This is a *compile-time* invalidation. Nothing happens at runtime — the machine code for a move is identical to a copy. **The safety is free.**

13.3 Copy versus move

Some types have no such problem: types living entirely on the stack, with no heap resource to double-free. These implement the `Copy` trait, and assignment duplicates them instead of moving.

```
let x = 5;
let y = x;
println!("{}", x);    // ✅ i32 is Copy
```

Copy types: all integers, `f32`/`f64`, `bool`, `char`, shared references `&T`, and tuples or arrays composed entirely of `Copy` types. `[i32; 5]` is `Copy`; `(i32, String)` is not.

Not Copy: `String`, `Vec<T>`, `Box<T>`, `&mut T`, and anything owning a resource or implementing `Drop`.

A type can never be both `Copy` and `Drop`; the compiler enforces this, for exactly the double-free reason above.

Shortcut: *if it touches the heap or manages a resource, it moves. If it is a plain stack scalar, it copies.*

13.4 Clone

```
let s1 = String::from("hello");
let s2 = s1.clone();    // allocates a NEW heap buffer and copies the bytes
println!("{s1} {s2}"); // ✅ two independent owners
```

`.clone()` is the equivalent of `copy.deepcopy()`. The design point is that **it is always explicit and always visible in the source**. Rust never silently deep-copies. When `.clone()` appears in a hot loop during profiling, you know exactly where the allocations came from — a diagnostic Python does not offer.

13.5 Ownership and functions

Passing a value to a function moves it, exactly as assignment does:

```
fn consume(s: String) {           // takes ownership
    println!("{s}");
}                                  // s dropped here

let s = String::from("hi");
consume(s);
println!("{s}");                  // ❌ s was moved into consume
```

Returning a value moves ownership back out:

```
fn produce() -> String {
    String::from("hi")           // ownership moves to the caller
}

fn pass_through(s: String) -> String {
    s                            // take it, give it back
}
```

That `pass_through` pattern — taking ownership solely to hand it back so the caller can keep using the value — is tedious and obviously unworkable as a general solution. This is precisely why borrowing exists.

14. Borrowing and References

A **reference** provides access to a value without taking ownership of it. Creating one is *borrowing*.

```
fn calculate_length(s: &String) -> usize {    // borrows
    s.len()
}                                              // s goes out of scope, but owns
                                              // nothing → no drop

let s1 = String::from("hello");
let len = calculate_length(&s1);
println!("{s1} has length {len}");           //  s1 still valid
```

`&s1` creates a reference to `s1`. No allocation, no move, no drop.

14.1 Mutable references

References are immutable by default, like everything else:

```
fn append(s: &mut String) {
    s.push_str(", world");
}

let mut s = String::from("hello");
append(&mut s);           // explicit &mut at the call site
println!("{s}");          // "hello, world"
```

Note that mutability is visible **at the call site** (`&mut s`), not only in the signature. Reading Rust, you can see which calls might mutate their arguments — something impossible to determine by inspection in Python.

14.2 The borrowing rules

At any given time you may have either any number of immutable references (`&T`) or exactly one mutable reference (`&mut T`) — never both.

References must always be valid — they may never outlive the data they point to.

This is usually shortened to "**shared XOR mutable**", and it is *the* central invariant of the language.

```
let mut s = String::from("hello");

let r1 = &s;           //  immutable borrow
let r2 = &s;           //  another immutable borrow — readers do not conflict
let r3 = &mut s;       //  cannot borrow `s` as mutable because it is also
                        //      borrowed as immutable

println!("{r1} {r2} {r3}");
```

14.3 Why the rule is worth the cost

It eliminates, at compile time:

- **Data races.** A race requires two accessors where at least one writes. Shared-XOR-mutable makes that structurally impossible. This is the entire basis of Rust's concurrency story (Part VII) — obtained for free from a rule you already had to follow.
- **Iterator invalidation.** In Python, mutating a list while iterating it produces silently wrong results. In Rust it does not compile, because iterating borrows the collection.
- **Unexpected aliasing.** The classic bug where a list is passed to a function, mutated, and a caller three frames up sees corrupted state. Rust makes this either impossible or explicitly declared in the signature.

It also enables **optimisation**: knowing that a `&mut T` is the *only* pointer to that data lets the compiler cache values in registers and reorder operations aggressively — guarantees a C compiler needs `restrict` `t` annotations to approach.

14.4 Non-lexical lifetimes

A borrow ends at its **last use**, not at the end of the enclosing block. This makes many obviously-correct programs compile:

```
let mut s = String::from("hello");

let r1 = &s;
println!("{r1}");           // last use of r1 — its borrow ENDS HERE

let r2 = &mut s;           //  no overlapping borrow
r2.push_str(" world");
```

If an older tutorial describes a snippet as an error but it compiles for you, non-lexical lifetimes are usually the reason.

14.5 No dangling references

```
fn dangle() -> &String {
    let s = String::from("hello");
    &s                // ✗ returns a reference to local data
}                    // s is dropped here – the reference would dangle
```

The C equivalent compiles happily and hands back a pointer to freed memory. Rust rejects it. The fix is to return the owned value and move it out:

```
fn no_dangle() -> String {
    String::from("hello")
}
```

15. Slices

A slice is a reference to a *contiguous run* of elements: a pointer plus a length, owning nothing.

```
let v = vec![1, 2, 3, 4, 5];
let all:    &[i32] = &v;           // the whole vector as a slice
let middle: &[i32] = &v[1..4];     // [2, 3, 4]

let s = String::from("hello world");
let hello: &str = &s[0..5];       // string slice
```

This is what `&str` actually is: **a slice into UTF-8 bytes**. Chapter 7 stated that `&str` borrows and `String` owns; now that borrowing has a definition, the whole distinction resolves.

Because a slice borrows, the borrow checker prevents the classic bug where a view outlives what it views:

```
let mut s = String::from("hello world");
let word = first_word(&s); // immutable borrow of s
s.clear();                 // ✗ needs &mut s while word still borrows
println!("{}", word);      // word would point into freed memory
```

Python has no defence against the equivalent bug. Rust makes it a compile error.

15.1 The API lesson: accept slices, return owned types

```
fn process(data: &str) { /* ... */ } //  accepts &String, &str, and literals
fn process(data: &String) { /* ... */ } //  needlessly restrictive
```

Thanks to **deref coercion**, `&String` automatically becomes `&str` and `&Vec<T>` becomes `&[T]` at call sites. A function taking `&str` or `&[T]` therefore accepts strictly more inputs, at zero cost.

Rule of thumb: parameters take `&str` and `&[T]`; return types and struct fields use `String` and `Vec<T>`.

For numerical work this is the everyday shape: `fn softmax(logits: &[f32]) -> Vec<f32>` — borrow the input, own the output.

16. Lifetimes

Every reference has a lifetime: the region of code over which it is valid. **You have been using lifetimes throughout this part** — the compiler inferred them. Annotations are needed only when inference is ambiguous.

16.1 The critical misconception

Lifetime annotations do not change how long anything lives. They do not extend, shorten, or manage anything. They are *descriptions of relationships that already exist*, written down so the compiler can verify them. Think of them as type-level documentation that the compiler checks — closer to a generic parameter than to any runtime mechanism.

16.2 When annotations are required

```
fn longest(x: &str, y: &str) -> &str { //  missing lifetime specifier
    if x.len() > y.len() { x } else { y }
}
```

The compiler's problem: the returned reference borrows from `x` or from `y`, and it cannot tell which. Without knowing, it cannot verify that the result does not outlive its source. You supply the relationship:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

Read `'a` as a generic lifetime parameter: *"for some lifetime `'a`, both inputs live at least that long, and the output is valid for exactly that long."* Concretely, `'a` resolves to the **shorter** of the two input lifetimes — the conservative, always-safe choice.

If a return value does not actually borrow from the inputs, no annotation is needed. Returning an owned `String` makes the question disappear entirely, and **while learning, that is often the right answer**.

16.3 Lifetime elision

The compiler applies three rules. If they fully determine every output lifetime, you write nothing:

1. Each elided **input** lifetime gets its own distinct parameter.
2. If there is **exactly one** input lifetime, it is assigned to **all** output lifetimes.
3. If one of the inputs is `&self` or `&mut self`, **self's lifetime** is assigned to all outputs.

Rule 2 is why `fn first_word(s: &str) -> &str` needs no annotation. Rule 3 is why methods almost never do. `longest` fails because it has two inputs (so rule 2 does not apply) and no `self` (so rule 3 does not apply).

16.4 Lifetimes in structs

A struct holding a reference must declare that it cannot outlive what it borrows:

```
struct Parser<'a> {
    input: &'a str,          // Parser cannot outlive the string it points into
    pos: usize,
}

impl<'a> Parser<'a> {
    fn rest(&self) -> &str {    // elision rule 3 handles this
        &self.input[self.pos..]
    }
}
```

This is a real and valuable pattern — zero-copy parsers and tokenizers are built on it — but structs holding references are a meaningful step up in complexity. **While learning, prefer owned fields** (`String`, `Vec<T>`). Reach for borrowed fields when profiling proves the copies matter.

16.5 `'static`

`'static` means "valid for the entire duration of the program". String literals are `&'static str`; they are baked into the binary.

```
let s: &'static str = "I live in the executable";
```

Two warnings:

1. `'static` is not a way to silence lifetime errors. If the compiler suggests it and the data is not genuinely program-long, the suggestion is wrong and you have a design problem.
2. The bound `T: 'static` means something subtler than "lives forever" — it means "contains no non-`'static` references", which `String` and `i32` satisfy trivially. This distinction becomes important with `thread::spawn` (Chapter 35).

17. Error Handling

Rust has **no exceptions**. There is no `try`/`except` and no invisible control flow that can unwind out of any line. Errors are **values**, returned from functions, and the type system requires you to acknowledge them.

This belongs alongside ownership because `Option` and `Result` are enums holding *owned* values — everything about moving, borrowing, and matching applies directly.

17.1 The two-way split

	Recoverable	Unrecoverable
Mechanism	<code>Result<T, E></code>	<code>panic!</code>
Meaning	"this can legitimately fail"	"a bug, or a broken invariant"
Python analogue	a caught exception	an assertion failure or crash
Example	file missing, bad input, network timeout	index out of bounds, violated invariant

17.2 `Option<T>`

```
enum Option<T> { Some(T), None }
```

Rust has no `null`. Absence is encoded in the type, so "forgetting to check for `None`" is a compile error rather than an `AttributeError: 'NoneType' object has no attribute ...` in production. This single design choice removes Tony Hoare's "billion dollar mistake".

```
let maybe: Option<i32> = Some(5);

match maybe {
    Some(n) => println!("got {n}"),
    None    => println!("nothing"),
}

if let Some(n) = maybe {           // when only one arm matters
    println!("got {n}");
}

let value = maybe.unwrap_or(0);    // supply a default
```

`match` is exhaustive: omitting the `None` arm does not compile.

17.3 `Result<T, E>`

```
enum Result<T, E> { Ok(T), Err(E) }
```

```
use std::fs::File;

let f = File::open("config.toml");    // Result<File, std::io::Error>

let f = match f {
    Ok(file) => file,
    Err(e)   => panic!("failed to open: {e}"),
};
```

`Result` is marked `#[must_use]`, so ignoring one produces a compiler warning. A failure cannot be silently swallowed the way a bare `except: pass` swallows an exception.

17.4 `unwrap` and `expect`

```
let f = File::open("config.toml").unwrap();
let f = File::open("config.toml").expect("config.toml missing");
```

Both panic on failure. They are appropriate in prototypes, examples, and tests.

In real code prefer `expect` with a message stating the invariant you believe holds. When it panics anyway, that message tells you which assumption broke. Treat `unwrap` in a production path as a code smell.

17.5 The `?` operator

```
use std::fs;
use std::io;

fn read_config() -> Result<String, io::Error> {
    let content = fs::read_to_string("config.toml")?; // Err → return early
    Ok(content.trim().to_string())
}
```

`?` means: if `Ok(v)`, unwrap to `v` and continue; if `Err(e)`, **return early** with that error. It is the ergonomic payoff that makes value-based error handling pleasant rather than verbose.

Two details worth knowing immediately:

1. `?` **converts error types automatically** via the `From` trait. If your function returns `MyError` and `?` encounters an `io::Error`, it calls `From::from` to convert — provided you implemented `From<io::Error> for MyError`. This is what makes error propagation across architectural layers clean.
2. `?` **also works on** `Option`, returning `None` early, inside a function that returns `Option`.

`?` works only in functions returning `Result` or `Option` — including `main`:

```
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let content = std::fs::read_to_string("config.toml")?;
    println!("{}", content);
    Ok(())
}
```

17.6 Combinators

```
maybe.map(|n| n * 2) // transform the Some/Ok value
maybe.and_then(|n| checked(n)) // chain another Option/Result-returning call
maybe.unwrap_or(0) // default value
maybe.unwrap_or_else(|| compute()) // lazily computed default
maybe.ok_or(MyError::Missing)? // Option → Result, then propagate
result.map_err(MyError::from)? // transform the error type
```

Coming from Python, these read as a cleaner version of chained `if x is not None` checks.

17.7 Defining your own errors

For **libraries**, define a concrete error enum so callers can match on its variants. The `thiserror` crate removes the boilerplate:

```
use thiserror::Error;

#[derive(Error, Debug)]
enum ConfigError {
    #[error("could not read config file")]
    Io(#[from] std::io::Error), // #[from] generates the From impl for ?
    #[error("invalid value for {key}")]
    BadValue { key: String },
}
```

For **applications**, where you mostly propagate and print, `anyhow` is the standard choice:

```
use anyhow::{Context, Result};

fn load() -> Result<Config> {
    let raw = std::fs::read_to_string("config.toml")
        .context("reading config.toml")?; // adds human-readable context
    Ok(parse(&raw?))
}
```

The convention: `thiserror` for libraries (typed and matchable), `anyhow` for binaries (ergonomic and contextual). `Box<dyn Error>` is the dependency-free middle ground.

17.8 When to panic

Panic when a **bug** has occurred — a broken invariant, an impossible state, a contract violation. Return a `Result` when failure is a **legitimate, expected outcome** that the caller should decide about.

A missing file is a `Result`. An index out of bounds in code you wrote to be in-bounds is a panic.

18. Working with the Borrow Checker

You will hit these errors. Recognising them by shape saves a great deal of time.

"cannot borrow `x` as mutable more than once" You are holding two `&mut` into the same data. Restructure so the borrows do not overlap in time (non-lexical lifetimes help), use `split_at_mut` for disjoint slice halves, or scope one borrow inside a `{ }` block.

"cannot borrow `x` as mutable because it is also borrowed as immutable" The classic case is iterating a collection while modifying it. Fixes: collect the changes into a `Vec` and apply them after the loop; iterate over indices instead of elements; or use in-place APIs such as `retain`, `iter_mut`, or `drain`.

"cannot move out of borrowed content" You have a `&T` and are trying to take ownership. Options: `.clone()` it; change the signature to take `T` by value; or use `std::mem::take(&mut x)` (which leaves `Default::default()` behind) or `std::mem::replace(&mut x, new)` to swap the value out.

"borrowed value does not live long enough" Something outlives its source. The usual fix is to return an owned value rather than a reference, or to hoist the owner into a longer-lived scope.

Two methods on `self` both want `&mut self` The borrow checker is field-sensitive *within* a function body but treats a method call as borrowing the whole struct. Fix by destructuring fields into locals first, or by splitting the struct so the two concerns own separate data.

18.1 Two pieces of pragmatic advice

`.clone()` is allowed while learning. Cloning to get past the borrow checker is not cheating; it trades a little performance for forward progress, and the result is still faster than the Python you are replacing. Get it compiling, then profile and remove the clones that matter. Fighting for zero-copy perfection on day three is how people give up.

When a design fights the borrow checker relentlessly, the design is usually wrong. Deeply interlinked object graphs — the kind Python encourages, where everything holds a reference to everything — are exactly what ownership rejects. The Rust answer is normally to restructure around ownership: a tree with clear parents, or indices into a central `Vec` instead of pointers between objects. `Rc<T>` and `RefCell<T>` (Chapter 25) exist as escape hatches for genuine shared ownership, but reaching for them in week two usually means avoiding the lesson.

18.2 Summary

Every value has exactly one owner; when the owner's scope ends, the value is dropped. Assigning or passing a heap-owning value **moves** it, invalidating the source, so nothing is ever freed twice. To use a value without taking it, **borrow** it — any number of shared `&` readers, or exactly one exclusive `&mut` writer, never both, and never outliving the data. **Lifetimes** name those relationships when the compiler cannot infer them. Errors are ordinary values — `Option<T>` for absence, `Result<T, E>` for failure — propagated with `?` and impossible to ignore silently.

Exercises — Part III

Do these in order; each targets one specific misconception.

1. **Feel the move.** Create a `String`, assign it to another variable, then print the first. Read the compiler error in full. Do the same with an `i32` and observe that it works. Explain the difference in a comment.
2. **Fix it three ways.** Take the broken snippet from exercise 1 and make it compile three different ways: with `clone`, with a reference, and by restructuring so ownership is returned. Note the cost of each.

3. **Ownership through functions.** Write `fn takes(s: String)` and `fn borrows(s: &String)`. Call each and observe which one lets you use the variable afterwards.
4. **Break the borrow rules deliberately.** Hold a `&` and a `&mut` to the same `String` simultaneously and read the error. Then move the `println!` of the immutable reference *above* the mutable borrow and watch non-lexical lifetimes make it compile.
5. **Iterator invalidation.** Try to `push` to a `Vec` inside a `for` loop over that same `Vec`. Fix it by collecting the new items into a separate `Vec` and extending after the loop. This is the Python bug Rust refuses to let you write.
6. **Write `first_word`.** `fn first_word(s: &str) -> &str`, returning the first whitespace-delimited word. Then call `s.clear()` while holding the result and understand the error.
7. **Lifetime annotation.** Implement `longest` yourself. Then try to return a reference to a `String` created *inside* the function, and understand why no annotation can rescue it.
8. **Elision check.** For each of these, decide whether elision applies *before* compiling: `fn f(x: &str) -> &str`, `fn g(x: &str, y: &str) -> &str`, `fn h(&self, x: &str) -> &str`.
9. **Option without null.** Write `fn find_user(id: u32) -> Option<String>`. Handle the result three ways: with `match`, with `if let`, and with `unwrap_or_else`.
10. **Error propagation.** Write a function that reads a file, parses its contents as an integer, and returns `Result<i32, Box<dyn Error>>`. Use `?` for both fallible steps, and note that two *different* error types propagate through the same operator.
11. **Custom error type.** Redo exercise 10 with a `thiserror` enum and `#[from]` conversions. Match on the variants at the call site.
12. **Zero-copy struct.** Build the `Parser<'a>` struct from §16.4. Then try to make it outlive the string it borrows and read the error carefully.

Part IV — Abstraction

Where the effort starts paying back. Part III taught the rules; this part covers the abstractions that make following them pleasant.

19. Generics

19.1 The organising idea: zero-cost abstraction

Bjarne Stroustrup's formulation, which Rust adopts wholesale:

What you do not use, you do not pay for. And what you do use, you could not hand-code any better.

In Python, abstraction costs you. A `for` loop over a generator, a call through an abstract base class, a `map()` — each goes through dynamic dispatch, allocates objects, and touches the interpreter. The abstraction is convenient and it is *slow*, which is why performance-critical Python means dropping into NumPy or C.

In Rust, the abstractions in this part compile away entirely. A ten-stage iterator chain becomes the same machine code as a hand-written loop. A generic function becomes a specialised copy per type, with no runtime type checks. A trait method call becomes a direct call.

You write high-level code and get assembly-level performance — which is exactly why `ruff`, `polars`, `uv`, and `tokenizers` are written in Rust.

Two mechanisms make this work, and understanding the split is most of this part:

- **Generics + traits** → **monomorphisation** → **static dispatch**. Resolved at compile time. Free.
- **Trait objects** (`Dyn Trait`) → **vtables** → **dynamic dispatch**. Resolved at runtime. Cheap, but not free.

You choose. Python chose for you, and it always chose the second.

19.2 Generic functions

```
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}
```

`<T: PartialOrd>` declares a type parameter `T` **bounded** by the `PartialOrd` trait. The bound is not decoration — without it, `item > largest` does not compile, because the compiler has no reason to believe an arbitrary `T` is comparable.

This is the fundamental contrast with Python's duck typing. The Python equivalent works on anything until it does not, and the failure surfaces at runtime, in production, on the one input type you did not test. Rust's version is verified at the *definition* site: if it compiles, it works for **every** type satisfying the bound. Rust generics are checked once, not per instantiation as C++ templates are.

19.3 Generic structs, enums, and impls

```
struct Point<T> {
    x: T,
    y: T,
}

impl<T> Point<T> {                // methods for ALL T
    fn x(&self) -> &T { &self.x }
}

impl Point<f64> {                // methods for ONLY Point<f64>
    fn distance_from_origin(&self) -> f64 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}
```

Note the `impl<T>` — the parameter is declared after `impl` so the compiler knows `T` is generic rather than a concrete type named `T`. The second block is a **specialised impl**: `distance_from_origin` exists only on `Point<f64>`. Calling it on a `Point<i32>` is a compile error, not a runtime one.

You have already been using generic enums extensively: `Option<T>` and `Result<T, E>` are exactly this.

19.4 Monomorphisation

At compile time, Rust generates a **separate concrete copy** of each generic item for every type it is used with:

```
let a = largest(&[1, 2, 3]);      // the compiler generates largest_i32
let b = largest(&["a", "b"]);    // and largest_str
```

No type parameter remains at runtime, no boxing, no lookup — just two ordinary functions that can be inlined and optimised independently. This is where the "zero cost" comes from.

The trade-off is binary size and compile time. Every instantiation is real code. This is a genuine reason Rust compiles more slowly than Go, and why heavily generic crates such as `serde` inflate build times. It is usually the right trade, but it is a trade.

19.5 `where` clauses

When bounds get long, move them out of the signature:

```
fn process<T, U>(t: &T, u: &U) -> String
    where
        T: Display + Clone,
        U: Clone + Debug,
    {
        format!("{t} {u:?}")
    }
```

Identical meaning, readable signature. Use `where` as soon as you have more than one or two bounds.

20. Traits

A trait defines shared behaviour. It is the closest analogue to a Python abstract base class or `typing.Protocol`, but checked at compile time and with no runtime cost.

```
trait Summary {
    fn summarize_author(&self) -> String;           // required

    fn summarize(&self) -> String {                 // default implementation
        format!("(Read more from {}...)", self.summarize_author())
    }
}

struct Tweet { username: String, content: String }

impl Summary for Tweet {
    fn summarize_author(&self) -> String {
        format!("@{}", self.username)
    }
    // summarize() comes free from the default
}
```

Default methods matter more than they first appear. A trait can be mostly implemented, requiring implementors to supply only the primitive operations. `Iterator` takes this to its logical extreme: you implement one method and receive around seventy-five.

20.1 Traits are decoupled from types

Unlike Python or Java, where a class declares its interfaces at the point of definition, **Rust lets you implement a trait for a type in an entirely separate place** — including implementing *your* trait for *standard library* types:

```
trait Doubled { fn doubled(&self) -> Self; }

impl Doubled for i32 {
    fn doubled(&self) -> i32 { self * 2 }
}

println!("{}", 21.doubled()); // extending a primitive, legally
```

This is powerful, scoped, and safe — and it is a large part of why Rust libraries compose so well. Unlike Python monkey-patching, it cannot affect code that does not import your trait.

20.2 The orphan rule

The obvious hazard: if two crates both implement `Display` for `Vec<T>`, which wins? Rust forbids the situation.

You may implement a trait for a type only if the trait or the type (or both) is local to your crate.

So you can implement your own trait for `Vec<i32>` ✓, or implement `Display` for your own struct ✓, but **not** `Display` for `Vec<i32>` ✗ — both are foreign.

The standard workaround is the **newtype pattern**: wrap the foreign type in a local one-field tuple struct.

```
struct Wrapper(Vec<String>);

impl std::fmt::Display for Wrapper {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "[{}]", self.0.join(", "))
    }
}
```

Newtypes cost nothing at runtime — the wrapper compiles away — and they are also the idiomatic way to add type safety. `struct Meters(f64)` and `struct Feet(f64)` become genuinely different types the compiler will not let you mix. For indexing-heavy work, `struct TokenId(u32)` versus `struct Position(u32)` prevents an entire class of bug.

20.3 Derivable traits

`#[derive(...)]` asks the compiler to generate a mechanical implementation:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
struct Config { dim: u32, layers: u32 }
```

Trait	Gives you	Notes
<code>Debug</code>	<code>{:?}</code> formatting	derive on nearly everything
<code>Clone</code>	explicit deep copy	required for <code>Copy</code>
<code>Copy</code>	implicit bitwise copy	only if all fields are <code>Copy</code>
<code>PartialEq</code> / <code>Eq</code>	<code>==</code>	<code>Eq</code> adds reflexivity; floats cannot be <code>Eq</code>
<code>PartialOrd</code> / <code>Ord</code>	<code><</code> , sorting	<code>Ord</code> is needed for <code>BTreeMap</code> keys
<code>Hash</code>	use as a <code>HashMap</code> key	requires <code>Eq</code> as well
<code>Default</code>	<code>Type::default()</code>	zero or empty per field

Deriving `Debug` on essentially every type you write is standard practice — `dbg!(&x)` and `{:?}` are the replacements for `print()`.

20.4 The standard traits worth knowing early

Trait	Purpose	Python analogue
<code>Display</code>	user-facing <code>{}</code> output	<code>__str__</code>
<code>Debug</code>	developer <code>{:?}</code> output	<code>__repr__</code>
<code>From<T></code> / <code>Into<T></code>	infallible conversion	constructors
<code>TryFrom<T></code> / <code>TryInto<T></code>	fallible conversion → <code>Result</code>	—
<code>Iterator</code>	produce a sequence	<code>__next__</code>
<code>IntoIterator</code>	can be iterated	<code>__iter__</code>
<code>Default</code>	a sensible zero value	default arguments
<code>Deref</code>	smart-pointer transparency	—
<code>Drop</code>	cleanup on scope exit	<code>__del__</code> / <code>__exit__</code>
<code>AsRef<T></code>	cheap reference conversion	—
<code>PartialEq</code> / <code>Ord</code>	comparison, sorting	<code>__eq__</code> / <code>__lt__</code>

Implement `From`, get `Into` free. This is a *blanket impl* in the standard library — `impl<T, U: From<T>> Into<U> for T`. Write one direction, get both:

```
impl From<Config> for String {
    fn from(c: Config) -> String { format!("{}", c.dim, c.layers) }
}

let s: String = config.into(); // works automatically
let s = String::from(config); // as does this
```

`From` is also what powers the `?` operator's automatic error conversion from Chapter 17.

20.5 Supertraits

A trait can require another:

```
trait Loggable: std::fmt::Display { // Loggable requires Display
    fn log(&self) { println!("[LOG] {self}"); }
}
```

Anything implementing `Loggable` must also implement `Display`, and `Loggable`'s default methods may rely on it.

21. Static and Dynamic Dispatch

This is the most consequential design decision in this part.

21.1 Static dispatch

```
fn notify(item: &impl Summary) { /* ... */ } // syntax sugar for:
fn notify<T: Summary>(item: &T) { /* ... */ } // exactly the same thing
```

Monomorphised, inlinable, zero cost. **This is the default choice.**

`impl Trait` also works in return position, where it means "some concrete type implementing this trait, and I am not naming it":

```
fn make_adder(x: i32) -> impl Fn(i32) -> i32 {
    move |y| x + y
}

fn evens(v: &[i32]) -> impl Iterator<Item = &i32> {
    v.iter().filter(|n| *n % 2 == 0)
}
```

The second example matters in practice: iterator adapter types are unwieldy to name (`Filter<Iter<'_, i32>, {closure}>`), and `impl Trait` lets you return them without doing so. The limitation is that all return paths must produce the **same** concrete type — you cannot return one iterator type from an `if` and a different one from the `else`.

21.2 Dynamic dispatch

When you genuinely need a heterogeneous collection — different concrete types behind one interface — you need a **trait object**:

```
let shapes: Vec<Box<dyn Shape>> = vec![
    Box::new(Circle { r: 1.0 }),
    Box::new(Square { side: 2.0 }),
];

for s in &shapes {
    println!("{}", s.area()); // resolved at runtime via a vtable
}
```

A `Vec<Box<dyn Shape>>` cannot be a `Vec<T>` because the elements have different sizes and layouts. `Box<dyn Shape>` is a **fat pointer**: one pointer to the data, one to a **vtable** of function pointers for that concrete type. Method calls go through the vtable.

This is precisely how *every* Python method call works. In Rust it is opt-in.

21.3 `dyn` compatibility

Not every trait can become a trait object. A vtable needs a fixed layout, so the trait must not have:

- methods that are **generic** over type parameters — each would need its own vtable entry, and there are unboundedly many;
- methods **returning** `Self` — the caller does not know the size;
- methods without a `self` receiver, i.e. associated functions;
- a `Sized` supertrait.

If the compiler says a trait "cannot be made into an object", this is why. The common fix is to split the `dyn`-compatible methods into their own trait.

*This property was called **object safety** until Rust 1.83 and is now called **dyn compatibility**. Older material uses the former term for the identical concept.*

21.4 Choosing

	Static (<code>impl Trait</code> / generics)	Dynamic (<code>dyn Trait</code>)
Resolved	compile time	runtime
Cost	zero; inlinable	pointer indirection; no inlining
Binary size	grows per instantiation	one copy
Compile time	slower	faster
Heterogeneous collections	✗	✓
Plugin / registry patterns	✗	✓

Default to static dispatch. Reach for `dyn` when you need runtime heterogeneity — a `Vec` of different implementations, a plugin system, or to cut monomorphisation bloat. The dispatch cost itself is real but modest; the lost inlining is usually the larger effect.

21.5 Associated types versus generic parameters

```
trait Container {  
    type Item; // associated type  
    fn get(&self, i: usize) -> Option<&Self::Item>;  
}  
  
trait Container2<T> { // generic parameter  
    fn get(&self, i: usize) -> Option<&T>;  
}
```

The difference is **how many times a type may implement the trait**:

- **Associated type** — once per implementing type. `Iterator` uses `type Item` because a `Vec<i32>`'s iterator yields exactly one thing, `i32`. Callers never have to specify it.
- **Generic parameter** — many times. `From<T>` is generic because a type can convert *from* many sources: `impl From<i32> for MyType`, `impl From<&str> for MyType`, and so on.

The rule: if there is one natural answer per type, use an associated type; if a type should support many, use a generic parameter.

22. Closures

Closures are anonymous functions that capture their environment — like Python lambdas, but unrestricted (multi-line, statements allowed) and integrated with ownership.

```
let factor = 3;
  let scale = |x: i32| x * factor;          // captures `factor` by reference
  println!("{}", scale(10));              // 30
```

Types are usually inferred, and each closure has its own unique anonymous type.

22.1 The three closure traits

How a closure captures determines which traits it implements:

Trait	Captures by	Callable	Rough Python analogue
<code>Fn</code>	<code>&T</code> (shared)	many times	a pure lambda
<code>FnMut</code>	<code>&mut T</code> (exclusive)	many times; needs <code>mut</code>	a lambda mutating a captured variable
<code>FnOnce</code>	<code>T</code> (by value, consuming)	once	a lambda that consumes what it closed over

These nest: every `Fn` is also `FnMut` and `FnOnce`; every `FnMut` is also `FnOnce`. The compiler infers the **least restrictive** one that works, so you rarely think about it until writing a function that *takes* a closure:

```
fn apply<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 { f(x) }
fn apply_mut<F: FnMut()>(mut f: F) { f(); f(); }
fn consume<F: FnOnce() -> String>(f: F) -> String { f() }
```

Take `FnOnce` if you will call it once, `FnMut` if repeatedly with mutation, `Fn` if repeatedly without. Being permissive in what you accept means `FnOnce` is the most flexible bound.

22.2 `move` closures

`move` forces capture by value:

```
let data = vec![1, 2, 3];
  let closure = move || println!("{}", data); // takes ownership of data
  // data is no longer usable here
```

This is essential when a closure outlives the scope that created it — when returning it, or when sending it to another thread (Chapter 35).

22.3 Async closures

Async closures were stabilised in Rust 1.85 and implement the `AsyncFn` family of traits, mirroring `Fn` / `FnMut` / `FnOnce`. They allow a closure to capture from its environment and return a future that borrows those captures — something an `async move` block could not previously express cleanly. See Chapter 41.

23. Iterators

23.1 The trait

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    // ...and roughly 75 default methods built on next()
}
```

You implement one method and get everything else free. This is the payoff of default methods from Chapter 20.

```
struct Fibonacci { a: u64, b: u64 }

impl Iterator for Fibonacci {
    type Item = u64;
    fn next(&mut self) -> Option<u64> {
        let out = self.a;
        self.a = self.b;
        self.b = out + self.b;
        Some(out)
    }
}

// instantly usable with the entire ecosystem:
let sum: u64 = Fibonacci { a: 0, b: 1 }.take(10).filter(|n| n % 2 == 0).sum();
```

23.2 Laziness

Iterator adapters do nothing until consumed. `map`, `filter`, and `take` merely build a new struct describing the work.

```
let v = vec![1, 2, 3];
v.iter().map(|x| { println!("side effect"); x * 2 }); // ⚠️ prints NOTHING
```

`Iterator` is `#[must_use]`, so the compiler warns here. To actually run the chain you need a **consumer**: `collect()`, `sum()`, `for_each()`, `count()`, a `for` loop, and so on.

The laziness has real consequences: a chain of ten adapters makes **one pass** over the data with no intermediate allocations, and LLVM typically fuses the whole thing into a single loop. Compare Python, where `[f(x) for x in xs if g(x)]` materialises lists and every step is interpreted.

23.3 The three ways to iterate

Method	Yields	Effect on the collection
<code>.iter()</code>	<code>&T</code>	borrow immutably; collection remains usable
<code>.iter_mut()</code>	<code>&mut T</code>	borrow mutably; lets you modify in place
<code>.into_iter()</code>	<code>T</code>	consumes the collection; yields owned values

```
let mut v = vec![1, 2, 3];

for x in v.iter()      { println!("{x}"); } // &i32
for x in v.iter_mut() { *x *= 2; }         // &mut i32 - modifies v
for x in v.into_iter() { println!("{x}"); } // i32 - v is GONE afterwards
```

And the `for`-loop sugar, which catches everyone once:

```
for x in &v      { } // == v.iter()
for x in &mut v  { } // == v.iter_mut()
for x in v      { } // == v.into_iter() - MOVES v
```

If you have ever written `for x in v` and then received "borrow of moved value" on the next line, this is why. It is Part III showing up in Part IV syntax.

23.4 The adapters you will use constantly

```
.map(|x| ...)           // transform each element
.filter(|x| ...)        // keep matching elements
.filter_map(|x| ...)     // transform and filter in one step (returns Option)
.enumerate()            // yields (index, item)
.zip(other)             // pair up two iterators
.take(n) / .skip(n)     // slicing
.take_while(|x| ...)    // stop at the first failure
.chain(other)           // concatenate
.rev()                 // reverse (requires DoubleEndedIterator)
.flatten()             // flatten nested iterables
.flat_map(|x| ...)      // map then flatten
.peekable()            // look ahead without consuming
```

Consumers:

```
.collect()              // build a collection
.sum() / .product()     // reduce numerically
.fold(init, |acc, x| ...) // general reduce (Python's functools.reduce)
.count() / .last()
.min() / .max() / .max_by_key(|x| ...)
.any(|x| ...) / .all(|x| ...)
.find(|x| ...) / .position(|x| ...)
.for_each(|x| ...)
```

Two slice methods worth flagging alongside these, because they are frequently what you actually want: `windows(n)` yields overlapping sliding windows, and `chunks(n)` yields non-overlapping fixed-size blocks. They are methods on slices rather than iterator adapters, and they cover n-gram extraction and batching respectively in a single call.

23.5 `collect()`

`collect()` builds any type implementing `FromIterator`, so it must be told which:

```
let v: Vec<i32>          = (1..=5).collect();
let s: String           = vec!['a', 'b'].into_iter().collect();
let set: HashSet<i32>    = v.iter().copied().collect();
let map: HashMap<&str, i32> = vec![("a", 1), ("b", 2)].into_iter().collect();

let v = (1..=5).collect::<Vec<i32>>(); // turbofish alternative
```

One of the best tricks in the standard library: an iterator of `Result`s collects into a `Result` of a collection, short-circuiting on the first error.

```
let nums: Result<Vec<i32>, _> = vec!["1", "2", "x"]
    .iter()
    .map(|s| s.parse::<i32>())
    .collect();

// Err(ParseIntError) - stops at "x"
```

The same works for `Option`. The Python equivalent requires an explicit loop with a `try`/`except`.

24. Collections

24.1 `Vec<T>`

Contiguous, heap-allocated, growable — the equivalent of a Python `list`.

```
let mut v: Vec<i32> = Vec::new();
let mut v = vec![1, 2, 3];
let v = Vec::with_capacity(1000);           // pre-allocate when the size is known

v.push(4);
v.pop();                                   // Option<T>
let x = v[0];                             // panics if out of bounds
let x = v.get(0);                          // Option<&T> - the safe way
v.sort();
v.sort_by_key(|x| x.abs());
v.retain(|x| *x > 0);                      // in-place filter
v.extend(other);
```

Growth is amortised $O(1)$ via capacity doubling, the same as Python's list. `with_capacity` avoids reallocation churn; in a hot loop over known-size data it is free performance.

The borrow rule bites here: you cannot hold `&v[0]` and then `push`, because a push may reallocate and invalidate the reference. Python has the same hazard and no protection against it.

24.2 `HashMap<K, V>`

```
use std::collections::HashMap;

let mut scores = HashMap::new();
scores.insert(String::from("blue"), 10);

let s = scores.get("blue"); // Option<i32>
let s = scores.get("blue").copied().unwrap_or(0);

for (k, v) in &scores { println!("{k}: {v}"); } // iteration order is RANDOM
```

Three things a Python developer should know:

1. **Iteration order is unspecified and randomised.** Unlike modern Python dicts, `HashMap` gives no insertion-order guarantee. If you need ordering, use `BTreeMap` (sorted by key) or the `indexmap` crate (insertion order).
2. **Keys must be `Eq + Hash`.** So `f64` cannot be a key, because `NaN != NaN` breaks the contract. This is the float-ordering note from Chapter 5 coming due.
3. **The default hasher is SipHash 1-3** — cryptographically strong and resistant to hash-flooding denial of service, but not the fastest available. For internal, non-adversarial workloads such as token counts and vocabulary maps, swapping in `rustc-hash` (`FxHashMap`) or `ahash` is often a 2–3× speedup on map-heavy code.

The **entry API** is the idiomatic replacement for `dict.setdefault` and `defaultdict`:

```
// word frequency count — the canonical example
let mut counts: HashMap<&str, i32> = HashMap::new();
for word in text.split_whitespace() {
    *counts.entry(word).or_insert(0) += 1;
}

map.entry(k).or_insert_with(Vec::new).push(item); // defaultdict(list)
map.entry(k).and_modify(|v| *v += 1).or_insert(1);
```

`entry` performs a single lookup for the check-and-insert, where the naive `if !map.contains_key(k) { map.insert(...) }` performs two.

24.3 The rest

Collection	Use when	Python analogue
<code>Vec<T></code>	default sequence	<code>list</code>
<code>VecDeque<T></code>	push and pop at both ends	<code>collections.deque</code>
<code>HashMap<K, V></code>	default key-value	<code>dict</code>
<code>HashSet<T></code>	membership, deduplication	<code>set</code>
<code>BTreeMap<K, V></code>	sorted keys, range queries	<code>sortedcontainers</code>
<code>BTreeSet<T></code>	sorted set	—
<code>BinaryHeap<T></code>	priority queue	<code>heapq</code>
<code>String</code>	owned UTF-8 text	<code>str</code>

`BTreeMap` is $O(\log n)$ rather than $O(1)$, but it keeps keys sorted and supports range queries (`map.range(10..20)`) — frequently worth the trade.

25. Smart Pointers and Interior Mutability

25.1 `Box<T>`

Heap allocation with single ownership. Required for three things:

1. **Trait objects** — `Box<dyn Trait>` (Chapter 21).
2. **Recursive types** — a type cannot contain itself directly, because its size would be infinite:

```
enum List {  
    Cons(i32, Box<List>),           // Box gives it a known size: one pointer  
    Nil,  
}
```

1. **Moving large values** without copying the bytes.

25.2 `Rc<T>`

Reference-counted shared ownership, single-threaded. Multiple owners; the value is freed when the last one drops.

This is exactly Python's object model, which is why it feels natural — and why over-reaching for it is a trap.

25.3 `RefCell<T>` and interior mutability

`RefCell<T>` allows mutation through a `&T`. It moves the borrow check from compile time to **runtime**; violating the rules panics rather than failing to compile.

```
use std::rc::Rc;
use std::cell::RefCell;

let shared = Rc::new(RefCell::new(vec![1, 2, 3]));
let clone = Rc::clone(&shared);           // cheap — bumps the refcount
clone.borrow_mut().push(4);               // mutate through a shared reference
```

`Rc<RefCell<T>>` is the "I want Python's object model" escape hatch. It works, it is safe — it panics rather than corrupting — and it costs refcount updates plus runtime borrow tracking.

Use it when the data genuinely is shared and mutable: graphs, observer patterns, trees with parent pointers. But if you are reaching for it in week two to dodge the borrow checker, restructure instead. The usual Rust answer is indices into a central `Vec` rather than pointers between objects.

Note also that `Rc` cycles **leak** — two nodes pointing at each other keep each other's refcount above zero forever. Break cycles with `Weak<T>`, which holds a non-owning reference that does not contribute to the count.

25.4 `Cow<'a, T>`

Clone-on-write: borrow if possible, clone only when mutation is actually needed. Excellent for text processing where most inputs pass through unmodified:

```
use std::borrow::Cow;

fn normalize(s: &str) -> Cow<str> {
    if s.contains(' ') { Cow::Owned(s.replace(' ', "_")) }
    else { Cow::Borrowed(s) }           // zero allocation in the common case
}
```

Worked example

Every concept in this part appears in the following program.


```

use std::collections::HashMap;

#[derive(Debug, Clone, PartialEq)]
struct Document { id: u32, text: String }

trait Tokenize {
    fn tokens(&self) -> Vec<&str>;

    fn token_count(&self) -> usize {           // default method
        self.tokens().len()
    }
}

impl Tokenize for Document {
    fn tokens(&self) -> Vec<&str> {
        self.text.split_whitespace().collect()
    }
}

/// Generic over anything tokenizable; static dispatch, zero cost.
fn vocabulary<'a, T: Tokenize>(docs: &'a [T]) -> HashMap<&'a str, usize> {
    let mut vocab = HashMap::new();
    for token in docs.iter().flat_map(|d| d.tokens()) {
        *vocab.entry(token).or_insert(0) += 1;
    }
    vocab
}

fn main() {
    let docs = vec![
        Document { id: 1, text: "the cat sat".into() },
        Document { id: 2, text: "the dog sat".into() },
    ];

    let vocab = vocabulary(&docs);

    // sort by frequency, descending – one iterator chain, single pass
    let mut ranked: Vec<_> = vocab.into_iter().collect();
    ranked.sort_by_key(|&(_, count)| std::cmp::Reverse(count));

    let total: usize = docs.iter().map(|d| d.token_count()).sum();

```

```
println!("{total} tokens, top: {:?}", ranked.first());
}
```

A trait with a default method, a generic function with a bound, a lifetime annotation tying output to input, iterator chaining with `flat_map`, the entry API, `collect` with inference, and a derive — all in thirty lines.

Exercises — Part IV

1. **Generic with a bound.** Write `fn largest<T: PartialOrd>(list: &[T]) -> &T`. Then remove the bound and read the error to see exactly what the bound buys you.
2. **Trait with a default.** Define a `Shape` trait with a required `area()` and a default `describe()` that uses it. Implement it for `Circle` and `Rectangle`.
3. **Static versus dynamic.** Write `fn print_area(s: &impl Shape)` and `fn print_all(shapes: &[Box<dyn Shape>])`. Explain why the second needs `Box<dyn>` and cannot be `&[impl Shape]`.
4. **dyn compatibility.** Add a method returning `Self` to `Shape` and watch `Box<dyn Shape>` stop compiling. Read the error, then fix it by splitting the trait.
5. **Orphan rule and newtype.** Try to `impl Display for Vec<String>`. Read the error. Fix it with a newtype wrapper.
6. **From and ?.** Implement `From<io::Error>` for a custom error enum, then write a function using `?` on an I/O operation. Confirm the conversion happens implicitly.
7. **Implement Iterator.** Write the `Fibonacci` iterator from §23.1. Then compute the sum of the first twenty even Fibonacci numbers in a single chain.
8. **The three iterations.** Take a `Vec<String>`. Iterate with `iter()`, then `iter_mut()` (uppercasing each in place), then `into_iter()`. After the third, try to use the vector and read the error.
9. **Laziness.** Write a chain with a `println!` inside `map` and do not consume it. Observe that nothing prints, and note the `must_use` warning. Add `.collect::<Vec<_>>()` and watch it run.
10. **Collect into Result.** Parse `vec!["1", "2", "3"]` into `Result<Vec<i32>, _>`, then change one element to `"x"` and observe the short-circuit.
11. **Word frequency.** Count word frequencies with the entry API, then return the top five by count. This is the canonical Rust exercise and it exercises half of this part.
12. **Closure traits.** Write three functions taking `Fn`, `FnMut`, and `FnOnce` respectively. Try passing a closure that consumes a captured `String` to each, and see which compile.
13. **Recursive type.** Define the cons-list enum from §25.1 without `Box` and read the "recursive type has infinite size" error. Then add `Box`.
14. **Shared mutation.** Build an `Rc<RefCell<Vec<i32>>>`, clone the `Rc`, and mutate through both handles. Then trigger a runtime borrow panic by holding a `borrow()` and a `borrow_mut()` simultaneously.

15. **Port something real.** Take a Python script of yours that does data preprocessing and port it. Note how much of the logic collapses into a single iterator chain.

Part V — Testing, Documentation, and Tooling

26. Testing

Rust's testing story is built into the language and requires no third-party framework.

26.1 Unit tests

Unit tests live in the same file as the code they test:

```
pub fn add(a: i32, b: i32) -> i32 { a + b }

#[cfg(test)]                                // only compiled during `cargo test`
mod tests {
    use super::*;                            // import the parent module

    #[test]
    fn adds_correctly() {
        assert_eq!(add(2, 2), 4);
    }

    #[test]
    #[should_panic(expected = "divide by zero")]
    fn panics_on_zero() { divide(1, 0); }

    #[test]
    fn works_with_results() -> Result<(), String> {
        let v = parse("42")?;
        assert_eq!(v, 42);
        Ok(())
    }
}
```

Because the tests are **inside the module**, they can test private functions — something Python makes awkward. `#[cfg(test)]` ensures the test code is entirely absent from release builds.

26.2 Integration tests

Integration tests go in a `tests/` directory at the project root. Each file is compiled as a separate crate and can use only your **public** API — which is exactly the point. They verify the interface your users actually see.

```
my_project/
├── src/lib.rs
└── tests/
    └── api.rs
```

26.3 Doc tests

Code examples in your documentation are compiled and run as tests. This is the feature no other mainstream language does as well.

```
/// Adds two numbers.
///
/// # Examples
///
/// ```
/// use my_crate::add;
/// assert_eq!(add(2, 2), 4);
/// ```
pub fn add(a: i32, b: i32) -> i32 { a + b }
```

`cargo test` runs that example. **Your documentation cannot rot**, because a stale example breaks the build.

26.4 Running tests

```
cargo test                # everything, in parallel
cargo test add            # only tests whose name matches "add"
cargo test -- --nocapture # show println! output
cargo test -- --test-threads=1 # run serially
```

26.5 Ecosystem additions

Need	Crate
Statistical benchmarking	<code>criterion</code>
Property-based testing	<code>proptest</code> , <code>quickcheck</code>
Snapshot testing	<code>insta</code>
Mocking	<code>mockall</code>

27. Documentation

```
//! Crate-level documentation – placed at the top of lib.rs.
//! Describes the whole module or crate.

/// Item-level documentation – placed above a function, struct, or trait.
/// Supports Markdown.
///
/// # Errors
/// Returns Err if the input is empty.
pub fn parse(s: &str) -> Result<i32, Error> { /* ... */ }
```

`cargo doc --open` generates a browsable site for your crate *and every dependency*.

Learn to read docs.rs. Every published crate has automatically generated documentation there, and it is typically far better than the average Python package's, because the tooling makes it automatic rather than optional. Every page has a "source" link that takes you straight to the implementation.

Conventional documentation section headings, which tooling and readers both expect:

Heading	Contents
<code># Examples</code>	Runnable examples; these become doc tests
<code># Errors</code>	The conditions under which <code>Err</code> is returned
<code># Panics</code>	The conditions under which the function panics
<code># Safety</code>	The invariants a caller must uphold for an <code>unsafe</code> function

28. Debugging and Diagnostics

```
RUST_BACKTRACE=1 cargo run      # full stack trace on panic
RUST_BACKTRACE=full cargo run
```

```
dbg!(&value);                      // prints file:line, the expression, and the value –
                                   // then returns the value, so it can be inserted inl
ine
eprintln!("{x:?}");                // write to stderr
```

For interactive debugging use `rust-gdb` or `rust-lldb`, or the CodeLLDB extension in VS Code, where breakpoints behave normally.

For structured logging in applications, `tracing` is the modern standard, with `log` plus `env_logger` as the simpler classic alternative.

29. Cargo in Depth

29.1 `Cargo.toml`

```
[package]
  name = "my_app"
  version = "0.1.0"
  edition = "2024"

[dependencies]
serde = { version = "1", features = ["derive"] }
tokio = { version = "1", features = ["full"] }

[dev-dependencies]
criterion = "0.5"      # test and benchmark only; not shipped

[profile.release]
lto = true              # link-time optimisation
codegen-units = 1       # slower build, faster binary
```

29.2 `Cargo.lock`

`Cargo.lock` records the exact resolved version of every dependency in the graph. `cargo new` tracks it in version control by default, and for **binaries and applications** you should keep it committed — it makes builds and deployments reproducible.

For **libraries**, the decision matters less than it is often claimed to, because `Cargo.lock` does not affect your consumers at all; only your `Cargo.toml` version requirements do. Committing it for a library gives you a reproducible CI baseline, at the cost of not automatically testing against the newest compatible dependency versions. The current Cargo guidance is that this is a project-by-project judgement rather than a rule, and that CI should verify against latest dependencies either way.

29.3 Features

Features are Cargo's conditional compilation mechanism: a crate ships optional functionality that you opt into. This is why you write `features = ["derive"]` for `serde`.

There is no direct Python equivalent; the nearest analogue is extras (`pip install pkg[extra]`), but Cargo features additionally control what code is compiled at all.

29.4 Workspaces

A workspace is a monorepo of related crates sharing one `Cargo.lock` and one `target/` directory:

```
[workspace]
members = ["core", "cli", "python-bindings"]
```

Shared builds mean a dependency is compiled once for the whole workspace rather than once per crate.

Exercises — Part V

1. **Unit tests on a private function.** Write a module with one public and one private function. Test both from a `#[cfg(test)] mod tests` block, and confirm the private one is reachable.
2. **Test a panic.** Write a function that panics on invalid input, and a `#[should_panic(expected = "...")]` test for it. Change the panic message and watch the test fail.
3. **Tests that return `Result`.** Convert an `unwrap`-heavy test to one returning `Result<(), Box<dyn Error>>` and using `?`.
4. **Integration test.** Move a function into `lib.rs`, then write a test in `tests/` that uses it. Make the function private and confirm the integration test stops compiling — this is the public/private boundary being enforced.
5. **Doc test.** Write a documented function with an `# Examples` section. Run `cargo test` and confirm the example executes. Then break the example deliberately and watch the build fail.
6. **Generate docs.** Run `cargo doc --open` on a project with at least two dependencies and browse the generated site, including the "source" links.

7. `dbg!` **in place**. Take an expression like `let x = compute(a) + compute(b);` and wrap one call in `dbg!` without restructuring the line.

Part VI — Memory Safety and the Edges of the Type System

30. What Memory Safety Means

"Rust is memory safe" is a claim with a precise meaning and precise limits. Both are worth stating directly.

30.1 The definition

Memory safety is the absence of undefined behaviour arising from memory access.

Concretely, seven bug classes:

Bug	What happens	Rust's defence
Use-after-free	Read or write memory that was freed	Ownership and the borrow checker: references cannot outlive their owner
Double-free	Free the same allocation twice	One owner, one <code>drop</code> ; moves invalidate the source
Dangling pointer	A pointer to memory that is gone	Lifetimes prove every reference is valid
Buffer overflow	Index past the end	Bounds checking on every index
Null dereference	Dereference a null pointer	There is no null — absence is <code>Option<T></code>
Uninitialised read	Read memory never written	The compiler requires initialisation before use
Data race	Concurrent unsynchronised access with at least one writer	<code>Send</code> / <code>Sync</code> plus shared-XOR-mutable (Part VII)

These are not exotic problems. **Roughly 70% of security vulnerabilities at Microsoft and Google have historically been memory-safety bugs.** That statistic is why Rust exists, why the NSA and CISA recommend memory-safe languages, and why both Linux and Windows have adopted Rust for new components.

The notable part is that all seven are caught **at compile time with no runtime cost** — except bounds checking, which is a highly predictable branch that LLVM frequently eliminates entirely when it can prove the index is in range.

30.2 What Rust does not protect you from

This is the part that gets oversold, so be clear-eyed about it.

Memory leaks are safe in Rust. Leaking is not undefined behaviour — it is merely wasteful. `Rc` reference cycles leak. `std::mem::forget` leaks deliberately. `Box::leak` returns a `&'static mut` on purpose. Rust guarantees you will not access memory you should not; it does not guarantee you will release it.

Deadlocks are safe. Two threads waiting on each other's locks is perfectly well-defined behaviour — and perfectly broken. Rust prevents data races, not deadlocks. Nothing stops you writing a lock-ordering bug.

Logic bugs are safe. The compiler verifies memory access, not correctness. An off-by-one in your algorithm compiles fine.

Integer overflow wraps in release builds. It panics in debug and wraps silently in release (Chapter 5). Not a memory-safety issue, but a real source of bugs.

Panics still crash. A `panic!` unwinds and terminates the thread. Safe, but the program still stops.

Race conditions are not data races. Rust eliminates data races — concurrent unsynchronised memory access. It does not eliminate logical races: two threads performing individually correct operations in a bad order still produce wrong answers.

***The accurate claim:** Rust eliminates undefined behaviour in safe code. It does not eliminate bugs. That is still an enormous win — undefined behaviour is what turns an ordinary bug into a security vulnerability — but it is not magic.*

30.3 The ownership-to-safety chain

Worth seeing all at once:

```
Ownership (one owner, scope-bound drop)
    → no double-free, no use-after-free, deterministic cleanup
Borrowing (shared XOR mutable)
    → no aliasing bugs, no iterator invalidation
Lifetimes (references cannot outlive data)
    → no dangling pointers
No null (Option<T>)
    → no null dereference
Bounds checking
    → no buffer overflows
Send + Sync
    → no data races
```

Every safety guarantee traces back to ownership. This is why Part III was the hard part and everything since has been comparatively easy.

31. `unsafe`

The most misunderstood keyword in the language. `unsafe` unlocks exactly **five** abilities:

1. Dereference a raw pointer (`*const T`, `*mut T`)
2. Call an `unsafe` function or method, including all FFI
3. Access or modify a mutable `static`
4. Implement an `unsafe` trait (such as `Send` or `Sync`)
5. Access the fields of a `union`

That is the complete list. Critically:

`unsafe` **does not turn off the borrow checker.** Ownership, lifetimes, and the borrowing rules apply identically inside an `unsafe` block. It permits only those five operations. It is not "C mode".

What `unsafe` actually means is: "I, the programmer, am asserting an invariant the compiler cannot verify." It shifts the burden of proof from the compiler to a human, for a small and auditable region of code.

```
let mut v = vec![1, 2, 3];
let ptr = v.as_mut_ptr();

unsafe {
    // SAFETY: index 1 is within the vector's length of 3.
    *ptr.add(1) = 99;
}
```

31.1 The safe-abstraction pattern

This is the reason the whole approach is workable. `Vec`, `String`, `Rc`, and `Mutex` are all built on `unsafe` internally and all expose a completely safe API. A few thousand carefully audited lines in the standard library allow millions of lines of application code to be safe. That is the entire strategy.

31.2 Practical guidance

You will likely never write `unsafe` outside of FFI. When you do:

- Validate invariants at the boundary, before entering the block.
- Keep the block as small as possible — `unsafe` the operation, not the function.
- Document the safety contract with a `// SAFETY:` comment stating *why* the invariant holds.
- Mark functions that impose requirements on callers as `unsafe fn`, with a `# Safety` doc section.
- Test under **Miri** (`cargo +nightly miri test`), which detects undefined behaviour at runtime.

32. Advanced Lifetimes

Chapter 16 covered annotations, elision, `'static`, and structs holding references. This chapter covers what was left out. Most of it is knowledge you need in order to *read* error messages rather than to write code.

32.1 Lifetime bounds

```
struct Wrapper<'a, T: 'a> {      // T must outlive 'a
    value: &'a T,
}

fn longest<'a, 'b: 'a>(x: &'a str, y: &'b str) -> &'a str {
    //      ^^^^^^^ 'b outlives 'a
    if x.len() > y.len() { x } else { y }
}
```

'b: 'a reads "'b outlives 'a" — anything valid for 'b is valid for at least 'a. T: 'a means "the type T contains no references shorter than 'a". The case you will actually encounter is T: 'static, which means something subtler than "lives forever":

T: 'static means "T contains no non-'static references."

String, i32, and Vec<u8> all satisfy T: 'static — they own their data and borrow nothing. &'a str does not. This distinction matters the moment you call thread::spawn, which requires 'static because the spawned thread may outlive the caller.

32.2 Higher-ranked trait bounds

```
fn apply<F>(f: F)
  where
    F: for<'a> Fn(&'a str) -> &'a str,      // for ALL lifetimes 'a
    { /* ... */ }
```

for<'a> means the closure must work for *every possible* lifetime, not one specific lifetime chosen by the caller. This is needed when passing closures that take references, because the lifetime is not known until the call site.

Mostly this is inferred and invisible. You will meet it explicitly when writing generic code over closures, and the error message "implementation is not general enough" is the tell.

32.3 Variance

Given 'long: 'short, is &'long T usable where &'short T is expected? Yes — this is **covariance**, and it is why you can pass a long-lived reference to a function wanting a short-lived one.

- &'a T is **covariant** in both 'a and T — longer works where shorter is wanted.
- &'a mut T is covariant in 'a but **invariant** in T — no substitution allowed.
- fn(T) is **contravariant** in T.
- Cell<T>, RefCell<T>, and Mutex<T> are invariant in T.

You will almost never reason about this deliberately. It is here so that when you hit a baffling lifetime error involving &mut, you know the word to search for. Invariance of &mut T is the usual culprit.

32.4 Lifetimes on trait objects

```
Box<dyn Trait>                // implicitly Box<dyn Trait + 'static>
Box<dyn Trait + 'a>           // explicitly allowed to borrow with lifetime 'a
&'a (dyn Trait + 'a)
```

Boxed trait objects default to `'static`. If your implementor holds references, you must say so explicitly — otherwise you get a confusing "does not live long enough" error on a type that appears to have no lifetimes at all.

32.5 Self-referential structs and `Pin`

A struct holding a reference into its own field is **impossible in safe Rust**:

```
struct SelfRef {  
    data: String,  
    slice: &str,    // ❌ pointing into self.data — no lifetime can express this  
}
```

The reason: moving the struct would move `data`, leaving `slice` dangling. Rust assumes all types are freely movable — a move is a `memcpy` — so this cannot be expressed.

`Pin<P>` is the answer: a wrapper asserting that the pointee will never move again. This is not something you use directly in application code, but it is **the reason async blocks work at all**, since an `async` block holding a reference across an `.await` is exactly a self-referential struct. When you see `Pin<Box<dyn Future>>` in async code, this is why.

32.6 Generic associated types

Stabilised in Rust 1.65 — associated types that themselves take generic parameters:

```
trait Container {  
    type Item<'a> where Self: 'a;  
    fn get<'a>(&'a self) -> Self::Item<'a>;  
}
```

This unlocks lending iterators — iterators yielding items that borrow from the iterator itself — and underpins ongoing async-trait improvements. This is library-author territory; know the name and move on.

33. Macros

Every `!` you have seen is a macro invocation: `println!`, `vec!`, `format!`, `panic!`, `assert!`, `dbg!`, `write!`, `matches!`.

Macros run at compile time and generate code, which is why `println!` can type-check its format string and accept a variable number of arguments — neither of which a Rust *function* can do.

You do not need to write macros for a long time, but you should know:

- `macro_rules!` defines *declarative* macros — pattern matching over syntax.
- **Procedural macros** are Rust programs that transform token streams. `#[derive(Debug)]` is one.
- `serde`, `thiserror`, `tokio::main`, and `clap` are all macro-driven. This is why they feel magical, and why their error messages are sometimes opaque.

Immediately useful macros:

```
dbg!(&x);                // prints file:line, the expression, and the value
assert_eq!(a, b);
assert!(cond, "message {x}");
matches!(x, Some(1..=5)); // a pattern test that evaluates to a bool
todo!()                  // typed placeholder that compiles and panics if reached
unimplemented!()
```

`todo!()` is genuinely useful while designing: it type-checks as any type, so you can write a full signature and fill in the body later without the compiler complaining about a missing return value.

34. Const Generics

Generics over *values*, not just types:

```
struct Matrix<const R: usize, const C: usize> {
    data: [[f32; C]; R],
}

fn dot<const N: usize>(a: [f32; N], b: [f32; N]) -> f32 { /* ... */ }
```

The dimensions are checked at compile time. For numerical and ML work this means **shape mismatches become compile errors** rather than runtime exceptions three hours into a training run.

This is not a day-one topic, but it is worth knowing it exists — it is a capability Python fundamentally cannot offer.

Exercises — Part VI

1. **Memory safety by demonstration.** Write the use-after-free, double-free, and iterator-invalidation bugs in Rust. Collect all three compiler errors. Then write the same three in Python and observe which ones it permits.

2. **Leaks are safe.** Build an `Rc` reference cycle — two structs pointing at each other via `Rc<RefCell<...>>`. Confirm it compiles, runs, and leaks. Then fix it with `Weak`.
3. **`unsafe` does not disable the borrow checker.** Write an `unsafe` block that still violates the borrowing rules. Confirm it fails to compile anyway.
4. **A safe abstraction.** Write a small wrapper around a raw pointer operation that exposes a completely safe API, with a `// SAFETY:` comment justifying the internal `unsafe`.
5. **`T: 'static`.** Write a function generic over `T: 'static` and try to pass it a `&str` borrowed from a local `String`. Read the error, then pass an owned `String` instead.
6. **Trait object lifetimes.** Create a `Box<dyn Trait>` from a type that holds a reference. Read the resulting error and fix it with `Box<dyn Trait + 'a>`.
7. **`todo!()` as a design tool.** Write a full trait with three method signatures, implement it with `todo!()` bodies, and confirm the whole thing type-checks before any logic exists.

Part VII — Concurrency and Async

This is where the ownership rules stop being a tax and start being a dividend.

35. Threads

35.1 Why concurrency is safe here

A **data race** requires three things simultaneously:

1. Two or more pointers to the same data
2. At least one of them writing
3. No synchronisation

You already banned items 1 and 2 in Part III — **shared XOR mutable**. That rule was not designed for concurrent code; it eliminates data races as a side effect. This is the design payoff of the entire language. In Python, the global interpreter lock papers over this by preventing true parallelism for CPU-bound work, and you *still* get race conditions with threads. In Go you get true parallelism and races are your problem, detected at runtime by `go run -race` if you are lucky. **In Rust, data races are compile errors.**

35.2 Spawning

```
use std::thread;

let handle = thread::spawn(|| {
    println!("from a thread");
    42
});

let result = handle.join().unwrap(); // wait, and get the value back
```

These are real OS threads, mapped 1:1, not green threads. `join()` returns a `Result` because the thread may have panicked — a panic in one thread does not kill the process.

Closures usually need `move`, because the thread may outlive the spawner:

```
let data = vec![1, 2, 3];
thread::spawn(move || println!("{data:?}")); // ownership transferred
// data is unusable here
```

`thread::spawn` requires `F: Send + 'static` — the closure must be transferable to another thread, and must not borrow anything from the current stack frame.

35.3 Scoped threads

Scoped threads (stable since Rust 1.63) relax the `'static` requirement, letting threads borrow locals because the scope guarantees they finish first:

```
let mut data = vec![1, 2, 3];

thread::scope(|s| {
    s.spawn(|| println!("{data:?}")); // borrows — no 'static needed
    s.spawn(|| println!("also {data:?}"));
}); // all threads joined here, guaranteed
```

This is considerably more ergonomic than wrapping everything in `Arc`, and it is underused by people who learned Rust before 1.63.

36. `Send` and `Sync`

These are **marker traits** — no methods, pure compile-time metadata.

Send — it is safe to **transfer ownership** of this type to another thread. **Sync** — it is safe to **share a reference** (`&T`) to this type across threads. Formally: `T: Sync` if and only if `&T: Send`.

Both are **auto traits**: the compiler implements them structurally. A struct is `Send` if all of its fields are `Send`. You almost never write these; you occasionally get told that a type is not one.

The instructive cases:

Type	Send	Sync	Why
<code>i32</code> , <code>String</code> , <code>Vec<T></code>	✓	✓	Plain owned data
<code>Rc<T></code>	✗	✗	Non-atomic refcount — concurrent clones would corrupt it
<code>Arc<T></code>	✓	✓	Atomic refcount (the <code>A</code>)
<code>RefCell<T></code>	✓	✗	Non-atomic borrow flag — safe to move, unsafe to share
<code>Mutex<T></code>	✓	✓	Synchronisation is its entire job
<code>MutexGuard<'_, T></code>	✗	✓	Must be released by the thread that locked
Raw pointers	✗	✗	No guarantees at all

Rc versus **Arc** is the canonical lesson. They are identical except that `Arc` uses atomic instructions for the refcount. `Arc` is marginally slower, so Rust makes you choose — and the compiler refuses to let you use `Rc` across threads. In Python this class of bug is invisible; here it is a compile error.

37. Shared State

```
use std::sync::{Arc, Mutex};
use std::thread;

let counter = Arc::new(Mutex::new(0));
let mut handles = vec![];

for _ in 0..10 {
    let counter = Arc::clone(&counter);          // cheap: a refcount bump
    handles.push(thread::spawn(move || {
        let mut n = counter.lock().unwrap();      // blocks until acquired
        *n += 1;
    }));                                          // guard dropped → unlocked
}

for h in handles { h.join().unwrap(); }
println!("{}", *counter.lock().unwrap());      // exactly 10, guaranteed
```

Read the type as two orthogonal problems solved by two composable types: `Arc` gives multiple owners across threads; `Mutex` ensures only one may access at a time.

The crucial design difference from C, Java, and Python:

***Rust's `Mutex` owns the data it protects.** You cannot access the data without locking, because the data is inside the mutex. The single most common concurrency bug in other languages — forgetting to take the lock — is structurally impossible.*

Unlocking is RAI: the `MutexGuard` releases the lock when it is dropped. **You cannot forget to unlock.** Compare Python, where `lock.acquire()` without a matching `release()` in a `finally` is a live hazard.

`lock()` returns a `Result` because of **poisoning**: if a thread panics while holding the lock, the protected data may be in a broken state, so subsequent lockers receive an `Err`. Calling `.unwrap()` here is normal — it propagates the failure.

`RwLock<T>` is the many-readers-or-one-writer variant. Use it when reads dominate writes; `Mutex` is otherwise simpler and often faster.

38. Message Passing

The alternative philosophy: *do not communicate by sharing memory; share memory by communicating.*

```
use std::sync::mpsc;

let (tx, rx) = mpsc::channel(); // multi-producer, single-consumer

for i in 0..3 {
    let tx = tx.clone();
    thread::spawn(move || tx.send(i * 10).unwrap());
}

drop(tx); // drop the original, or rx never ends

for received in rx { // iterates until all senders are dropped
    println!("{received}");
}
```

Ownership makes channels genuinely safe: `send` **moves the value**, so the sender provably cannot touch it afterwards. The "I sent it but also kept using it" bug does not exist.

For anything serious, use `crossbeam-channel` — multi-consumer, faster, and with `select` support.

When to use which: channels for pipelines and worker pools, where ownership flows in one direction and is easy to reason about; `Arc<Mutex<T>>` for genuinely shared mutable state such as a cache or a counter. Prefer channels when both would work.

39. Atomics

Lock-free primitives for simple values:

```
use std::sync::atomic::{AtomicUsize, Ordering};

static COUNTER: AtomicUsize = AtomicUsize::new(0);
COUNTER.fetch_add(1, Ordering::Relaxed);
```

`Ordering` controls memory-barrier strength: `Relaxed`, `Acquire`, `Release`, `AcqRel`, `SeqCst`.

Use `SeqCst` **unless you have measured a need and understand the memory model**. This is genuinely subtle territory, where being clever produces bugs that appear once a week on one CPU architecture.

40. Data Parallelism with Rayon

```
use rayon::prelude::*;

let sum: f64 = data.par_iter().map(|x| expensive(x)).sum();
//          ^^^ that is the entire diff
```

Work-stealing data parallelism across all cores, with the borrow checker guaranteeing correctness. Every iterator chain from Chapter 23 has a parallel twin: `par_iter`, `par_iter_mut`, `into_par_iter`, `par_chunks`, `par_sort`.

For preprocessing pipelines this is the highest value-per-keystroke in the language. There is no Python equivalent — `multiprocessing` means serialising data across process boundaries, and threads are throttled by the GIL.

41. Async

41.1 Concurrency versus parallelism

- **Parallelism** — doing many things *simultaneously* on multiple cores. Bound by CPU. → **threads / rayon**
- **Concurrency** — managing many things *in flight*, most of them waiting. Bound by I/O. → **async**

Ten thousand open sockets, mostly idle: ten thousand OS threads costs on the order of 80 GB of stack and overwhelms the scheduler. Ten thousand async tasks costs a few megabytes. That is the entire value proposition.

Rule: CPU-bound → threads or rayon. I/O-bound with high concurrency → async. Do not reach for async because it sounds modern; it is a specific tool for a specific bottleneck.

41.2 Rust ships no runtime

```
async fn fetch(url: &str) -> Result<String, Error> {
    let resp = request::get(url).await?;
    Ok(resp.text().await?)
}
```

An `async fn` does not return a `String` — it returns `impl Future<Output = Result<String, Error>>`. And here is what surprises every Python developer:

The language defines `async` / `await` and the `Future` trait. It provides no executor. There is no built-in event loop. You must choose a runtime.

Python bundles `asyncio`. Rust deliberately does not, because an embedded device, a web server, and a database need very different schedulers. In practice this means `tokio` — the de facto standard, and the right default unless you know why not — or `smol` and `embassy` for lighter or embedded cases.

```
[dependencies]
tokio = { version = "1", features = ["full"] }
```

```
#[tokio::main]                                // macro: builds a runtime, blocks on main
async fn main() {
    let body = fetch("https://example.com").await.unwrap();
    println!("{body}");
}
```

41.3 Futures are lazy

```
let fut = fetch(url);           // NOTHING has happened yet
let body = fut.await;           // now it runs
```

A `Future` is an inert state machine until it is polled. Calling an async function does zero work.

Python coroutines behave similarly; **JavaScript promises do not** — they start immediately. If your mental model came from JavaScript, adjust it.

The consequence: `async fn` bodies do not run "in the background" merely because you called them. For background execution you must **spawn**:

```
let handle = tokio::spawn(async move { fetch(url).await }); // runs now
let result = handle.await.unwrap();
```

41.4 How it works underneath

The compiler transforms each `async` block into a **state machine** implementing `Future`:

```
trait Future {
    type Output;
    fn poll(self: Pin<&mut Self>, cx: &mut Context) -> Poll<Self::Output>;
}

enum Poll<T> { Ready(T), Pending }
```

Each `.await` is a **suspension point** — a state in the machine. When polled, the future runs until it reaches an `.await` that is not ready, returns `Pending`, and registers a `Waker`. When the I/O completes, the waker tells the executor to poll again, resuming exactly where it left off.

Two things follow:

1. **The state machine holds live variables across `.await` points.** If one of them is a reference into another, the future is self-referential — hence `Pin` (§32.5). That is the entire reason `Pin` exists.
2. **Async tasks are cheap.** No OS thread and no separate stack — just an enum whose size is that of its largest state. Tasks are hundreds of bytes, not megabytes.

41.5 Running things concurrently

```
// Sequential — 2 seconds total
let a = fetch(url1).await;
let b = fetch(url2).await;

// Concurrent on ONE task — 1 second
let (a, b) = tokio::join!(fetch(url1), fetch(url2));

// Concurrent AND parallel, across threads
let h1 = tokio::spawn(fetch(url1));
let h2 = tokio::spawn(fetch(url2));
let (a, b) = (h1.await?, h2.await?);

// Race — first to finish wins, the others are dropped
tokio::select! {
    r = fetch(url) => println!("{r:?}"),
    _ = tokio::time::sleep(Duration::from_secs(5)) => println!("timeout"),
}

// Many at once
let results = futures::future::join_all(urls.iter().map(|u| fetch(u))).await;
```

`join!` multiplexes on one task; `spawn` hands work to the runtime's thread pool. `select!` is how timeouts and cancellation are implemented.

41.6 The three pitfalls

1. **Never block inside async.** This is the big one.

```
async fn bad() {
    std::thread::sleep(Duration::from_secs(1)); // ❌ blocks the whole worker thread
    let data = std::fs::read_to_string("x")?;   // ❌ blocking I/O
    expensive_cpu_work();                       // ❌ starves every other task
}
```

An async task that blocks stops *every other task* on that worker thread. The fixes:

```
tokio::time::sleep(d).await; // ✅ async sleep
tokio::fs::read_to_string("x").await?; // ✅ async I/O
tokio::task::spawn_blocking(|| expensive()).await?; // ✅ offload CPU work
```

For an inference server, the model forward pass is CPU- or GPU-bound and **must** go through `spawn_blocking` or a dedicated thread pool, or request handling collapses under load. This is the most common real-world async Rust mistake.

2. Do not hold a `std::sync::MutexGuard` across `.await`.

```
let guard = mutex.lock().unwrap();
something().await; // ❌ guard is not Send → the future is not Send
d
```

The compiler catches this with a confusing "future is not `Send`" error. Fix it by scoping the lock so it ends before the `.await`, or use `tokio::sync::Mutex` when you genuinely must hold it across a suspension point — it is slower, so prefer restructuring.

3. Dropping a future cancels it. Unlike Python, where a `Task` is cancelled explicitly, in Rust dropping a future stops it at its last suspension point — mid-operation, with no cleanup callback. This makes `select!` dangerous with operations that are not **cancellation-safe**: a partially read stream can lose data. Read the documentation on cancellation safety before using `select!` in a loop.

41.7 Practical async

Async traits. `async fn` in traits was stabilised in Rust 1.75, but with limitations — such methods are not `dyn`-compatible, and `Send` bounds are awkward to express. For trait objects, use the `async-trait` crate:

```
#[async_trait::async_trait]
trait Fetcher {
    async fn fetch(&self, url: &str) -> Result<String>;
}
```

Streams — the async equivalent of `Iterator`, not yet in the standard library. Use `futures::Stream` or `tokio_stream`:

```
use tokio_stream::StreamExt;
while let Some(item) = stream.next().await { /* ... */ }
```

This is the mechanism for streaming LLM tokens back to a client.

Structured concurrency — `JoinSet` manages a dynamic group of tasks and cancels them on drop:

```
let mut set = tokio::task::JoinSet::new();
for url in urls { set.spawn(fetch(url)); }
while let Some(res) = set.join_next().await { /* ... */ }
```

Function colouring. Async functions can only be `.await`ed from async contexts. This "colouring" splits a codebase in two and is a genuine ergonomic cost, identical to Python's. Design your layering deliberately: keep pure logic synchronous and push async to the I/O edges.

42. Choosing the Right Tool

Situation	Use
CPU-bound work over a collection	<code>rayon</code> (<code>par_iter</code>)
A few long-running background jobs	<code>std::thread</code>
Threads that need to borrow locals	<code>thread::scope</code>
Thousands of network connections	<code>tokio</code> (async)
HTTP or gRPC server	<code>axum</code> / <code>tonic</code> (async)
Shared counter or cache across threads	<code>Arc<Mutex<T>></code> or <code>Arc<RwLock<T>></code>
A pipeline of stages	Channels (<code>crossbeam-channel</code>)
A simple shared counter	<code>AtomicUsize</code>
Model inference inside a web server	Async handler + <code>spawn_blocking</code> for the forward pass

A **typical inference server** uses `tokio` and `axum` for request handling, `spawn_blocking` or a dedicated thread pool for model execution, channels for batching requests together, and `rayon` for parallel preprocessing. All four in one binary, each doing what it is good at.

42.1 What is still your problem

Rust guarantees the absence of data races. It does **not** prevent:

- **Deadlocks** — always acquire locks in a consistent global order.
- **Livelock and starvation.**
- **Logical race conditions** — individually correct operations in the wrong order.
- **Holding a lock too long** — the classic scalability killer.

Exercises — Part VII

1. **Rc across threads.** Try to send an `Rc<i32>` to a spawned thread. Read the `Send` error in full. Swap to `Arc` and watch it work.
2. **The counter.** Build the `Arc<Mutex<i32>>` ten-thread counter. Then remove the `Mutex` and try to share `Arc<i32>` mutably — see what the compiler says.
3. **Scoped threads.** Rewrite exercise 2 with `thread::scope`, borrowing the counter instead of wrapping it in `Arc`.
4. **Channels.** Build a producer/consumer: three producer threads and one consumer summing the results. Then forget to drop the original `tx` and watch it hang — and understand why.
5. **Rayon.** Take a CPU-heavy iterator chain from Part IV, add `par_`, and time both under `--release`. Then try to mutate a shared `Vec` inside `par_iter` without a lock and read the error.
6. **Async laziness.** Call an `async fn` without `.await` and confirm nothing happens. Add `.await`. Then `tokio::spawn` it.
7. **Sequential versus concurrent.** Write three async functions that each sleep one second. Time them awaited in sequence, then with `join!`. Confirm three seconds versus one.
8. **Block the executor.** Put `std::thread::sleep` inside an async task alongside nine others and watch throughput collapse. Fix it with `tokio::time::sleep`, then again with `spawn_blocking`.
9. **Guard across await.** Hold a `std::sync::MutexGuard` across an `.await` in a spawned task. Read the "future is not `Send`" error. Fix it two ways: by scoping the lock, and with `tokio::sync::Mutex`.
10. **Timeout with `select!`.** Race a slow operation against `tokio::time::sleep`. Then consider what state the cancelled operation was left in.

Part VIII — Practice

43. Performance

Things that actually matter once you are optimising, in rough order of impact.

1. **Build with `--release`.** Covered in Chapter 2; it dwarfs everything else on this list.
2. **Avoid allocation in hot loops.** `String::new()` or `vec![]` inside a loop is the usual culprit. Hoist buffers out and reuse them with `.clear()`; use `with_capacity` when the size is known.
3. **Prefer borrowing to cloning** — but only after profiling says so.
4. **Swap the hasher.** `FxHashMap` or `AHashMap` in place of the DoS-resistant default, for internal maps.
5. **Iterator chains are already optimal.** They compile to the same code as manual loops. Do not hand-unroll; you will usually make it worse.

6. `Vec<T>` **beats linked structures** almost always — cache locality dominates asymptotics at realistic sizes.
7. **Profile, do not guess.** `cargo flamegraph` and `perf` for profiles, `criterion` for microbenchmarks, `hyperfine` for whole-program timing.
8. **Tune the release profile.** `lto = true`, `codegen-units = 1`, and `panic = "abort"` are typically worth 10–20% in exchange for a slower build.

Never micro-optimize before profiling. Rust makes it tempting because the control is right there in front of you. Resist — the compiler is smarter than your intuition about it.

44. Python-Developer Gotchas

44.1 No default arguments, keyword arguments, or overloading

Rust has none of these. This surprises people more than the borrow checker does. The four workarounds:

```
// 1. Option parameters
fn connect(host: &str, timeout: Option<u64>) { /* ... */ }

// 2. A config struct plus Default
fn connect(cfg: Config) { /* ... */ }
connect(Config { host: "x".into(), ..Default::default() });

// 3. The builder pattern — idiomatic when there are many options
Client::builder().host("x").timeout(30).build()?;

// 4. Differently named constructors instead of overloads
Vec::new(); Vec::with_capacity(10); Vec::from(slice);
```

Budget for this in API design.

44.2 No inheritance

There is no `class Foo(Bar)`. Rust has traits, which provide shared behaviour with defaults, and composition, where structs contain other structs. Any design leaning on deep inheritance hierarchies must be restructured.

In practice this is a feature — "composition over inheritance" enforced by the language rather than recommended by a style guide.

44.3 No REPL

There is no `python -i`. Use play.rust-lang.org for snippets, `cargo new scratch` for local experiments, or install `evcxr` for an actual REPL. Expect this to feel limiting at first; the compiler's immediate feedback partly replaces it.

44.4 Integer division and modulo differ

Rust truncates toward zero; Python floors.

```
Rust:  -7 / 2 == -3      -7 % 2 == -1      (remainder; sign follows the dividend)
Python: -7 // 2 == -4    -7 % 2 == 1       (floor; sign follows the divisor)
```

This is a genuine source of silent bugs when porting algorithms. Use `div_euclid` and `rem_euclid` for Python semantics.

44.5 No `**` operator

Use `x.pow(2)` for integers, and `x.powi(2)` or `x.powf(0.5)` for floats.

44.6 String formatting

```
println!("{}", x);           // inline capture (1.58+) – prefer this
println!("{}", x);           // positional
println!("{x:?}");           // Debug
println!("{x:#?}");          // pretty Debug – excellent for structs
println!("{x:.2}");           // two decimal places
println!("{x:>10}");           // right-align, width 10
let s = format!("{}", x);     // build a String instead of printing
```

44.7 Compile times are genuinely slow

This is Rust's most legitimate criticism. Mitigate with `cargo check`, `rust-analyzer`, keeping dependency counts sane, and `sccache` for shared caching.

44.8 `clone()` is not a moral failure

Clone freely while learning and profile later. Idiomatic code that was never written is worth nothing.

45. The Ecosystem

Rust's standard library is deliberately small; the ecosystem fills the gaps. Knowing the canonical crate for each job saves an enormous amount of time.

Need	Crate	Python analogue
Serialisation (JSON and more)	<code>serde</code> + <code>serde_json</code>	<code>json</code> , <code>pydantic</code>
Async runtime	<code>tokio</code>	<code>asyncio</code>
HTTP client	<code>reqwest</code>	<code>requests</code>
Web server	<code>axum</code> (or <code>actix-web</code>)	<code>fastapi</code>
CLI parsing	<code>clap</code> (derive API)	<code>argparse</code> , <code>typer</code>
Data parallelism	<code>rayon</code>	<code>multiprocessing</code>
Error handling	<code>thiserror</code> (library), <code>anyhow</code> (application)	<code>exceptions</code>
Logging and tracing	<code>tracing</code>	<code>logging</code>
Regular expressions	<code>regex</code>	<code>re</code>
Dates and times	<code>chrono</code> or <code>jiff</code>	<code>datetime</code>
Randomness	<code>rand</code>	<code>random</code>
Extra iterator adapters	<code>itertools</code>	<code>itertools</code>
Benchmarking	<code>criterion</code>	<code>pytest-benchmark</code>
Property testing	<code>proptest</code>	<code>hypothesis</code>
Faster hashing	<code>rustc-hash</code> , <code>ahash</code>	—
Environment and config	<code>dotenvy</code> , <code>config</code> , <code>figment</code>	<code>python-dotenv</code>

`serde` **deserves special mention.** It is derive-driven, near-zero-cost, and the single most important crate in the ecosystem:

```
#[derive(Serialize, Deserialize, Debug)]
struct Config { name: String, layers: u32 }

let cfg: Config = serde_json::from_str(&raw)?;           // parse, validate, and type-check
k

let json = serde_json::to_string(&cfg)?;
```

That is `pydantic`-grade validation with no runtime cost, checked at compile time. For most Python developers it is the moment Rust starts to feel *better* rather than merely faster.

Find crates via lib.rs, which is better curated, or crates.io. Judge candidates by recent release activity, download count, and the quality of their docs.rs pages.

46. Numerical, Data, and ML Work

Need	Crate
N-dimensional arrays	<code>ndarray</code> (NumPy-shaped API)
Linear algebra	<code>nalgebra</code> (small fixed-size, graphics-flavoured)
DataFrames	<code>polars</code>
Tokenization	<code>tokenizers</code> (Hugging Face, Rust-native)
ONNX inference	<code>ort</code> (ONNX Runtime bindings)
PyTorch bindings	<code>tch-rs</code> (libtorch)
Model weights	<code>safetensors</code>
Data parallelism	<code>rayon</code>
GPU compute	<code>wgpu</code> , <code>cudarc</code>
Python bindings	<code>pyo3</code> + <code>maturin</code>

Three things worth understanding early:

Memory layout is now yours to control. A contiguous `Vec<f32>` with manual striding beats `Vec<Vec<f32>>` decisively — the nested version scatters rows across the heap and destroys cache locality. This is why `ndarray` and every real tensor library use flat buffers with separate shape metadata.

`rayon` is nearly free parallelism. Changing `.iter()` to `.par_iter()` parallelises a chain with compile-time-verified correctness. For CPU-bound preprocessing this is frequently a 4–8× win for a one-word diff.

SIMD. `std::simd` (portable SIMD) is still nightly-only and has unresolved design questions blocking stabilisation. On stable, use the `wide` crate, rely on autovectorisation — which LLVM does well when you write simple loops over slices — or use architecture-specific intrinsics, most of which have been safe to call since Rust 1.87.

The realistic near-term win is not rewriting your models; it is rewriting the *bottleneck*. Tokenization, data loading, feature extraction, custom loss computation. Wrap it with PyO3, ship it as a wheel, and keep the rest in Python. That is the pattern `tokenizers` and `polars` themselves follow.

47. A Learning Path

Stage	Content
0 — Setup (day 1)	<code>rustup</code> , Cargo basics, rust-analyzer, clippy, rustfmt, edition 2024, debug-versus-release awareness (Part I)
1 — Foundations (week 1)	Data types, functions, control flow, structs, enums, pattern matching, modules (Part II)
2 — Workflow (week 1)	Testing, documentation, <code>dbg!</code> , reading compiler errors (Part V)
3 — Ownership (weeks 2–4)	Ownership, borrowing, lifetimes, error handling (Part III)
4 — Abstraction (weeks 4–6)	Traits, generics, closures, iterators, collections, smart pointers (Part IV)
5 — Ecosystem	<code>serde</code> , <code>clap</code> , <code>anyhow</code> / <code>thiserror</code> , <code>rayon</code> — build a real CLI at this point
6 — Concurrency	Threads, <code>Send</code> / <code>Sync</code> , <code>Arc<Mutex<T>></code> , channels, async and <code>tokio</code> (Part VII)
7 — Advanced	<code>unsafe</code> , macros, const generics, advanced lifetimes, profiling (Part VI)
8 — Applied	PyO3 and maturin, <code>ndarray</code> , <code>polars</code> internals, <code>tokenizers</code> (Chapter 46)

Two observations that are easy to miss:

The plateau is real and is not a sign of failure. There is a stretch — usually weeks two through four — where you understand every rule individually and still cannot satisfy the compiler. Everyone hits it. The way through is writing code, not reading more explanations. If you are stuck on a borrow error for more than twenty minutes, `.clone()` it, add a `// TODO: remove clone`, and move on. You will understand the fix in two weeks; today you need momentum.

The single highest-leverage habit is running `cargo clippy` on everything you write, starting on day one. It teaches idiomatic Rust faster than any book, because it corrects your actual code rather than describing someone else's.

48. Further Reading

Primary references

Resource	What it is
The Rust Programming Language	The official book. The canonical source; read it end to end at least once.
Rust by Example	Runnable snippets organised by topic.
The Standard Library Docs	Searchable, with a "source" link on every item.
The Rust Reference	The precise language specification, for when the book is not exact enough.
The Cargo Book	Manifest format, features, profiles, workspaces.
The Rustonomicon	<code>unsafe</code> Rust, in depth. Read only when you need it.

The book is also available offline: `rustup doc --book`.

Practice

- `rustlings` — small compiler-driven exercises; the best possible companion to Part II and Part III.
- **Rust Playground** — for snippets, and for sharing reproductions when asking for help.

Reading other people's code

More than in most languages, Rust has a strong idiom culture that books do not fully convey. Reading real code is how you absorb it. Good targets, in rough order of accessibility:

- `ripgrep` — a clean, well-structured, realistic CLI application.
- `tokenizers` — directly relevant if your work is ML-adjacent.
- `polars` — large, sophisticated, production-scale.
- **The standard library source** — click "source" on any docs.rs or std page. It is far more readable than you would expect.

Video

Jon Gjengset's "Crust of Rust" series is the strongest intermediate Rust material available, and is particularly good on the topics in Part VI.

Appendix A — Python to Rust Quick Reference

A.1 Values and types

Concept	Python	Rust
Default mutability	Mutable	Immutable (<code>let</code>); opt in with <code>mut</code>
Reuse a name with a new type	Reassignment	Shadowing (<code>let</code> again)
Integers	One unbounded <code>int</code>	Twelve sized types; default <code>i32</code> ; index type <code>usize</code>
Integer overflow	Never (arbitrary precision)	Panics in debug, wraps in release; use explicit methods
Float default	<code>float</code> (= f64)	<code>f64</code>
Truthiness	Yes	None — <code>if</code> requires a real <code>bool</code>
Character	A one-character <code>str</code>	<code>char</code> — a 4-byte Unicode scalar
Tuple access	<code>t[0]</code>	<code>t.0</code> (compile-time)
Array	Dynamic <code>list</code>	Fixed <code>[T; N]</code> ; dynamic is <code>Vec<T></code>
String	One <code>str</code>	<code>&str</code> (borrowed) versus <code>String</code> (owned); both UTF-8
Numeric conversion	Implicit	Explicit — <code>as</code> , <code>From</code> , <code>TryFrom</code>
Constant	<code>UPPER = ...</code> by convention	<code>const</code> / <code>static</code> ; type annotation required
Integer division	Floors	Truncates toward zero (<code>div_euclid</code> for Python behaviour)
Exponentiation	<code>x ** 2</code>	<code>x.pow(2)</code> , <code>x.powi(2)</code> , <code>x.powf(0.5)</code>

A.2 Memory and ownership

Concept	Python	Rust
Memory management	Garbage collector, at runtime	Ownership, at compile time; no GC
<code>b = a</code>	Two names, one object	Move (or copy, if <code>Copy</code>)
Deep copy	<code>copy.deepcopy()</code>	<code>.clone()</code> — always explicit
Pass to a function	Passes a reference implicitly	Moves , unless you write <code>&</code>
Read-only access	Convention only	<code>&T</code> — enforced
Mutable access	Any reference can mutate	<code>&mut T</code> — exclusive, and visible at the call site
Aliasing rule	Anything goes	Many <code>&T</code> XOR one <code>&mut T</code>
Cleanup	GC; <code>with</code> for resources	Scope-based <code>Drop</code> (RAII), automatic
Dangling pointer	Impossible (GC)	Impossible (borrow checker)
Memory leaks	Possible (cycles)	Possible — and <i>safe</i>
Escape hatch	C extensions	<code>unsafe</code> (five specific powers)

A.3 Errors

Concept	Python	Rust
Null	<code>None</code> , checked at runtime	No null ; <code>Option<T></code> , checked at compile time
Errors	Exceptions, invisible control flow	<code>Result<T, E></code> values, <code>?</code> to propagate
Ignoring an error	<code>except: pass</code>	<code>#[must_use]</code> warning; must be handled
Unrecoverable failure	Uncaught exception	<code>panic!</code>
Adding context	Exception chaining	<code>anyhow::Context</code> , <code>map_err</code>

A.4 Abstraction

Concept	Python	Rust
Interface	ABC / <code>Protocol</code> , at runtime	<code>trait</code> , at compile time, zero cost
Adding methods to a builtin	Monkey-patching	<code>impl MyTrait for i32</code> — safe and scoped
Generic function	Duck typing; fails at runtime	<code><T: Bound></code> ; verified at the definition
Generic cost	Dynamic everything	Monomorphised; free
Heterogeneous list	Native	<code>Vec<Box<dyn Trait>></code> — opt-in
Dispatch	Always dynamic	Static by default; <code>dyn</code> on request
Inheritance	<code>class Foo(Bar)</code>	None — traits plus composition
Lambda	<code>lambda</code> (expression only)	Closure (full body); <code>Fn</code> / <code>FnMut</code> / <code>FnOnce</code>
Iteration protocol	<code>__iter__</code> / <code>__next__</code>	<code>IntoIterator</code> / <code>Iterator</code>
Lazy sequences	Generators	All iterators are lazy
Comprehension	<code>[f(x) for x in xs if g(x)]</code>	<code>xs.iter().filter(g).map(f).collect()</code>
Iterator cost	Interpreted; allocates	Fused into one loop; no allocation
<code>dict</code>	Insertion-ordered	<code>HashMap</code> unordered; <code>BTreeMap</code> sorted
<code>defaultdict</code>	<code>defaultdict(int)</code>	<code>.entry(k).or_insert(0)</code>
<code>reduce</code>	<code>functools.reduce</code>	<code>.fold(init, f)</code>
Conversion	<code>int()</code> , <code>str()</code>	<code>From</code> / <code>Into</code> , <code>TryFrom</code> / <code>TryInto</code>
<code>__str__</code> / <code>__repr__</code>	Dunder methods	<code>Display</code> / <code>Debug</code>
Default arguments	Native	Config struct, builder, or <code>Option</code>

A.5 Concurrency

Concept	Python	Rust
True parallelism	Blocked by the GIL	Native threads; real
Data races	Possible and silent	Compile error
Deadlocks	Possible	Possible
Shared ownership	Automatic	<code>Arc<T></code> — explicit
Lock discipline	<code>with lock:</code> ; forgettable	<code>Mutex<T></code> owns the data
Unlocking	Manual or context manager	Automatic, when the guard drops
Data parallelism	<code>multiprocessing</code> (IPC cost)	<code>rayon</code> (<code>par_iter</code>)
Async runtime	<code>asyncio</code> , built in	Bring your own (<code>tokio</code>)
Coroutine laziness	Lazy	Lazy
Task cost	~KB	~hundreds of bytes
Cancellation	<code>task.cancel()</code>	Drop the future
Blocking in async	Slows the loop	Starves the worker thread

A.6 Tooling

Need	Python	Rust
Package manager	<code>pip</code> , <code>poetry</code> , <code>uv</code>	<code>cargo</code>
Lockfile	<code>poetry.lock</code>	<code>Cargo.lock</code>
Formatter	<code>black</code>	<code>cargo fmt</code> (rustfmt)
Linters	<code>ruff</code> , <code>pylint</code>	<code>cargo clippy</code>
Test runner	<code>pytest</code>	<code>cargo test</code> (built in)
Doc generator	<code>sphinx</code> , <code>makedocs</code>	<code>cargo doc</code> (built in)
Type checker	<code>mypy</code> (optional)	The compiler (mandatory)
Optional dependencies	Extras	Cargo features
Monorepo	—	Workspaces

Appendix B — Glossary

Associated type — A type placeholder in a trait, fixed once per implementing type. `Iterator::Item` is the canonical example.

Auto trait — A trait the compiler implements structurally, based on a type's fields. `Send` and `Sync` are auto traits.

Blanket impl — An implementation covering all types matching a bound, such as `impl<T: Display> ToString for T`.

Borrow — Access to a value without ownership, via a reference. Shared (`&T`) or exclusive (`&mut T`).

Crate — A compilation unit: one library or one binary.

`dyn` compatibility — The property that allows a trait to be used as a trait object. Formerly called *object safety*.

Deref coercion — The automatic conversion of `&String` to `&str` and `&Vec<T>` to `&[T]` at call sites.

Drop — The automatic cleanup that runs when a value's owner goes out of scope.

Edition — An opt-in set of language changes (2015, 2018, 2021, 2024). Set per crate; editions interoperate.

Elision — The compiler's inference of lifetime annotations from three fixed rules.

Fat pointer — A pointer carrying extra metadata: a length for slices, a vtable pointer for trait objects.

Interior mutability — Mutating data through a shared reference, with the borrow rules checked at runtime. `RefCell<T>`, `Cell<T>`, `Mutex<T>`.

Lifetime — The region of code over which a reference is valid. Annotations describe existing relationships; they do not change them.

Monomorphisation — Generating a separate concrete copy of a generic item for each type it is used with. The source of Rust's zero-cost generics.

Move — The transfer of ownership from one binding to another, invalidating the source.

Newtype — A single-field tuple struct wrapping another type, used to work around the orphan rule or to add type safety.

NLL (non-lexical lifetimes) — The rule that a borrow ends at its last use rather than at the end of its block.

Orphan rule — You may implement a trait for a type only if the trait or the type is local to your crate.

Package — One `Cargo.toml`: at most one library crate plus any number of binaries.

Panic — An unrecoverable error that unwinds and terminates the thread.

RAII — *Resource Acquisition Is Initialization*: tying resource cleanup to scope exit.

Shared XOR mutable — The central borrowing invariant: many shared references, or one exclusive reference, never both.

Slice — A borrowed view into a contiguous run of elements: `&[T]`, `&str`.

Trait object — A `dyn Trait` value behind a pointer, dispatched at runtime via a vtable.

Turbofish — The `::<T>` syntax that supplies a type argument explicitly, as in `parse::()`.

Unit type — `()`, the zero-sized "no meaningful value" type. The return type of functions that return nothing.

Workspace — A set of related packages sharing one `Cargo.lock` and one build directory.

Zero-cost abstraction — An abstraction that compiles to code no worse than the equivalent hand-written version.

End of document.